

Linear Regression



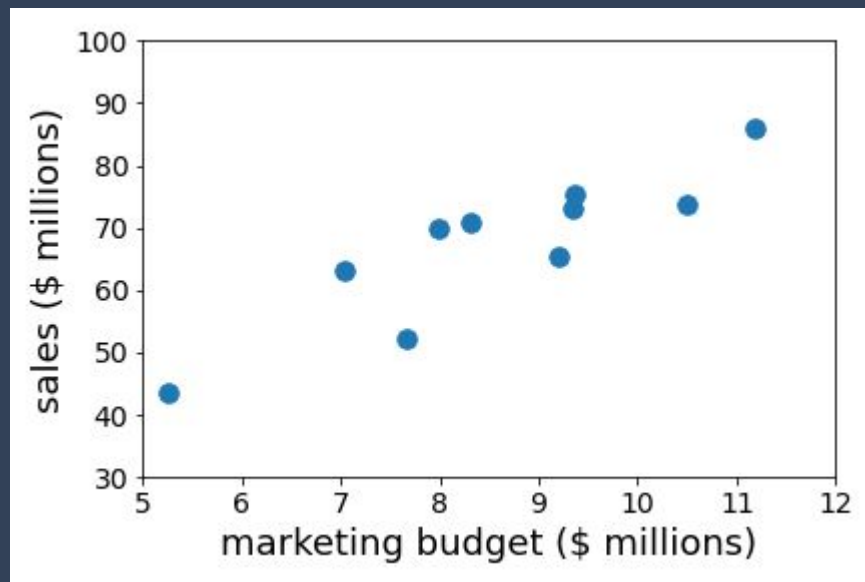
INTERVIEW
KICKSTART

Problem Statement

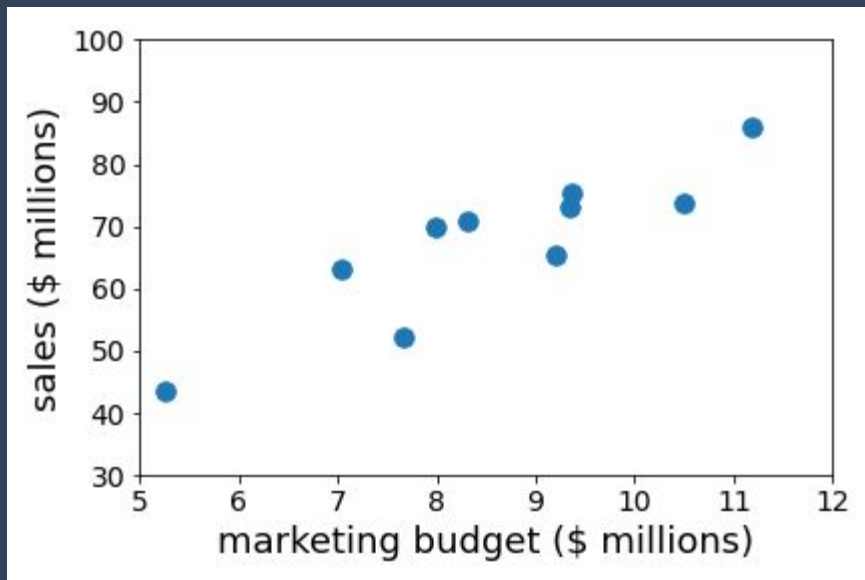
Budget (x)	Sales (y)
9.36	75.25
5.26	43.48
10.50	73.82
9.35	73.05
9.2	65.38
...	...
7.67	52.07

Problem Statement

Budget (x)	Sales (y)
9.36	75.25
5.26	43.48
10.50	73.82
9.35	73.05
9.2	65.38
...	...
7.67	52.07



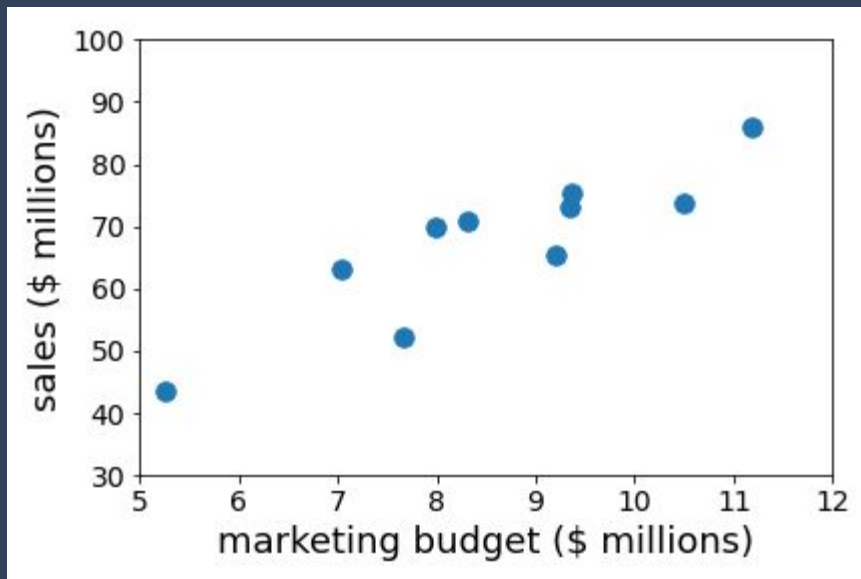
Intuition



Intuition

One “straight” line to describe the relation of y and x

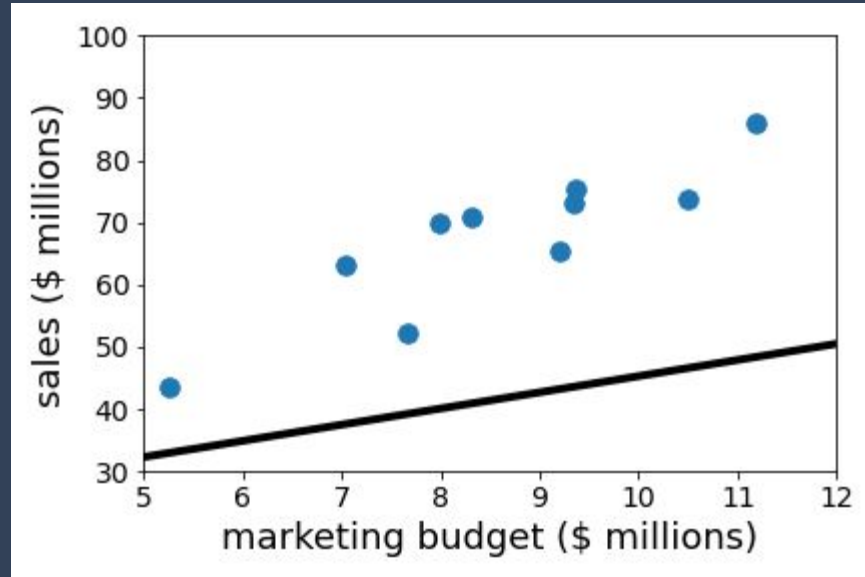
$$y = w_1 * x + w_2$$



Intuition

One “straight” line to describe the relation of y and x

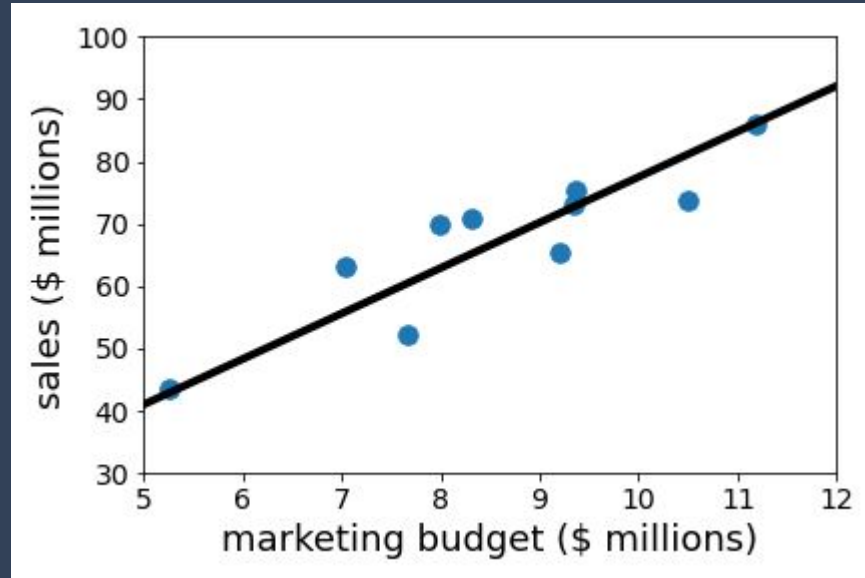
$$y = w_1 * x + w_2$$



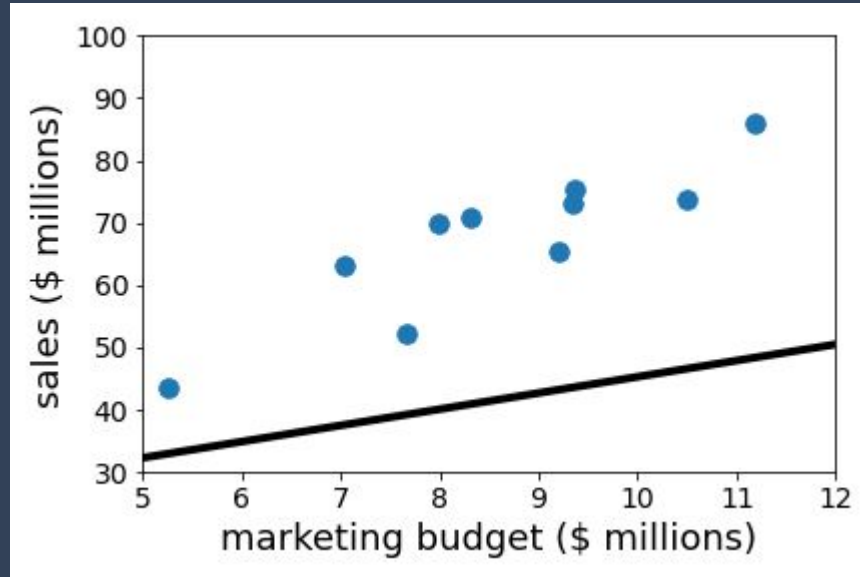
Intuition

One “straight” line to describe the relation of y and x

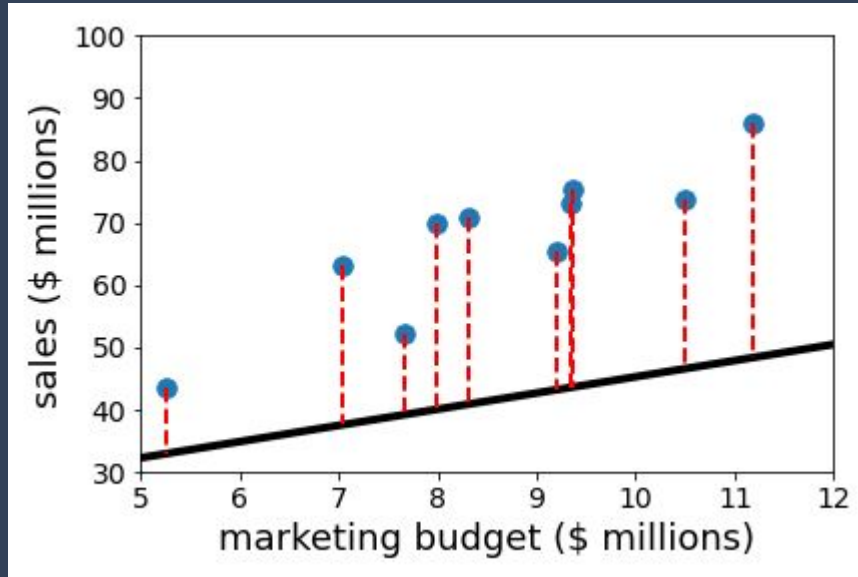
$$y = w_1 * x + w_2$$



Mean Absolute Error (MAE)

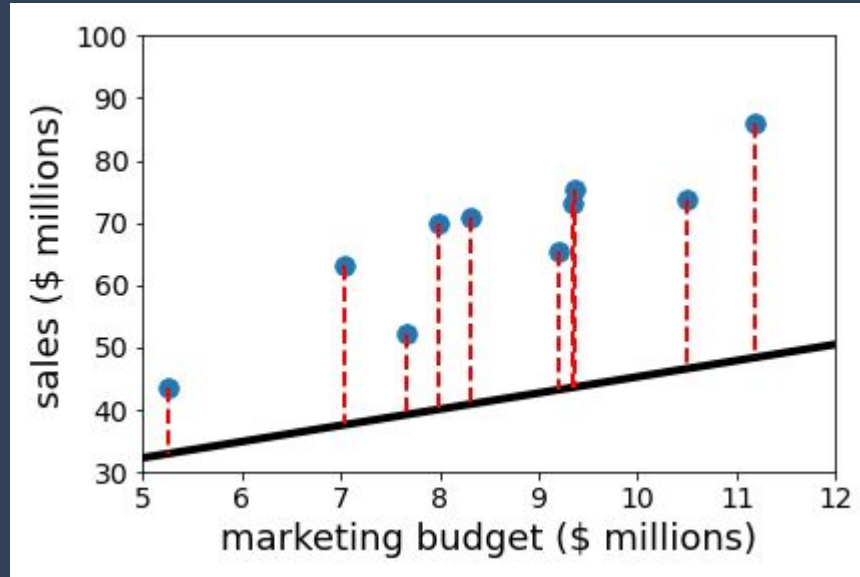


Mean Absolute Error (MAE)

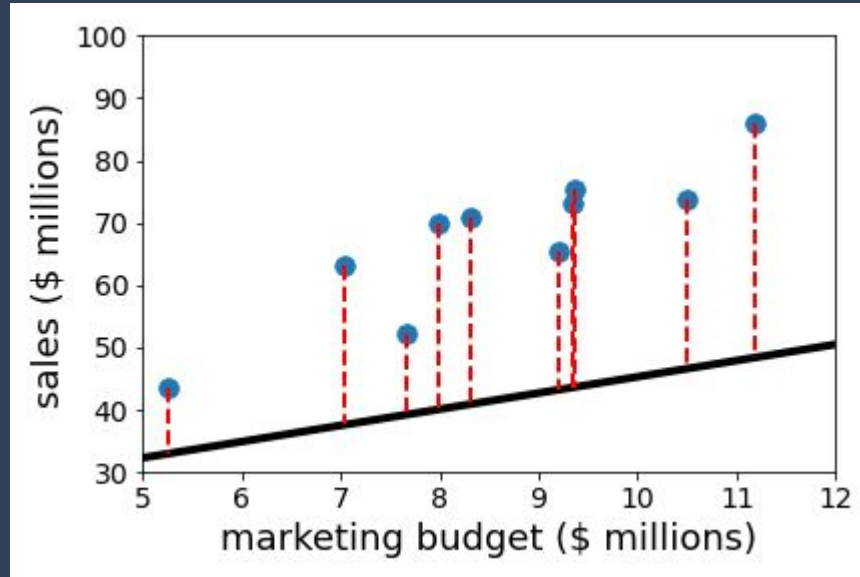


Mean Absolute Error (MAE)

$$Error = \frac{1}{m} \sum_{i=1}^m |y - \hat{y}|$$

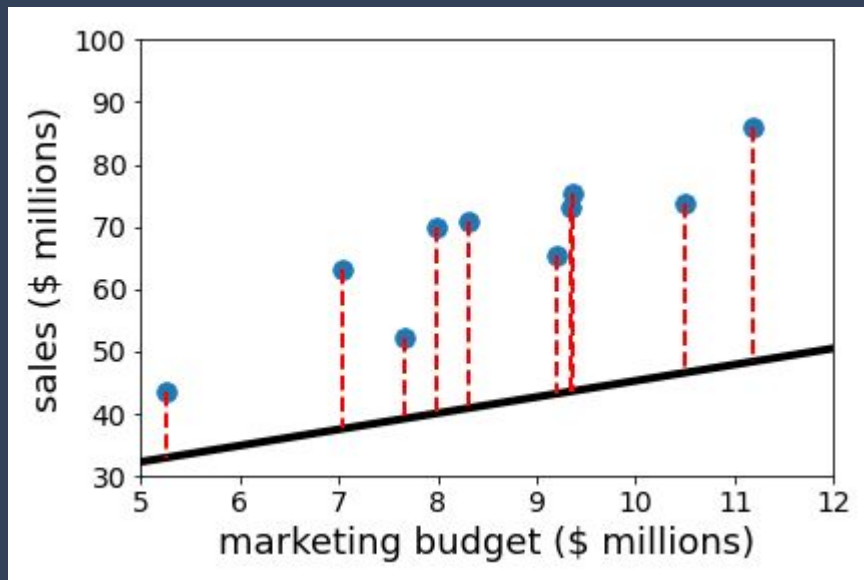


Mean Squared Error (MSE)



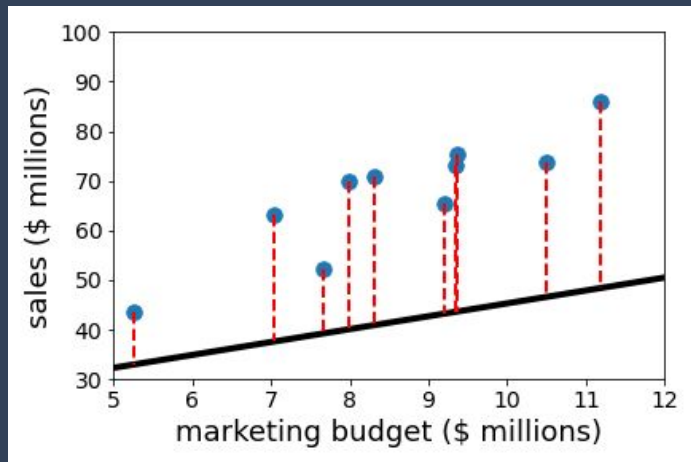
Mean Squared Error (MSE)

$$Error = \frac{1}{2m} \sum_{i=1}^m (y - \hat{y})^2$$



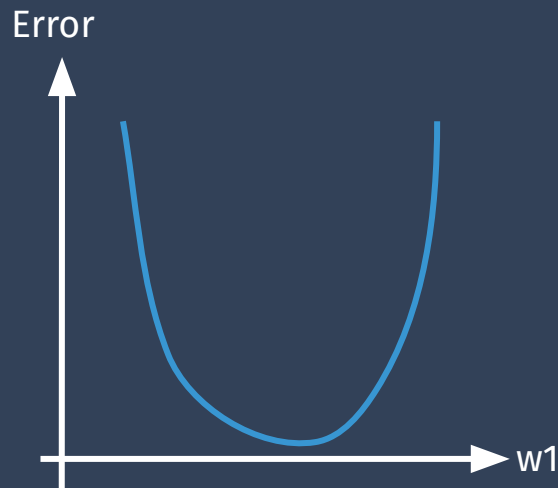
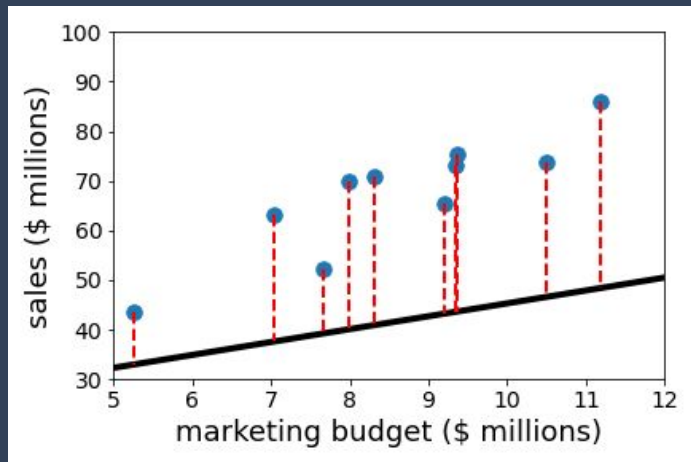
Move the line

$$Error = \frac{1}{2m} \sum_{i=1}^m (y - \hat{y})^2$$



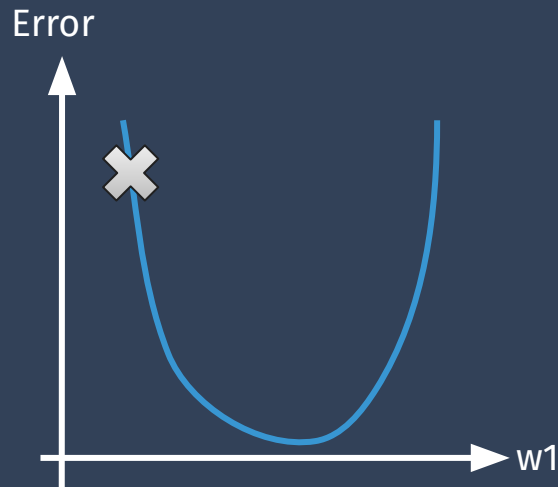
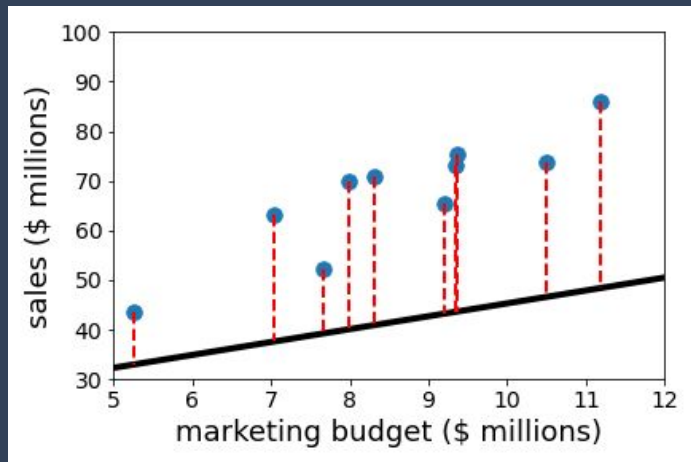
Move the line

$$Error = \frac{1}{2m} \sum_{i=1}^m (y - \hat{y})^2$$



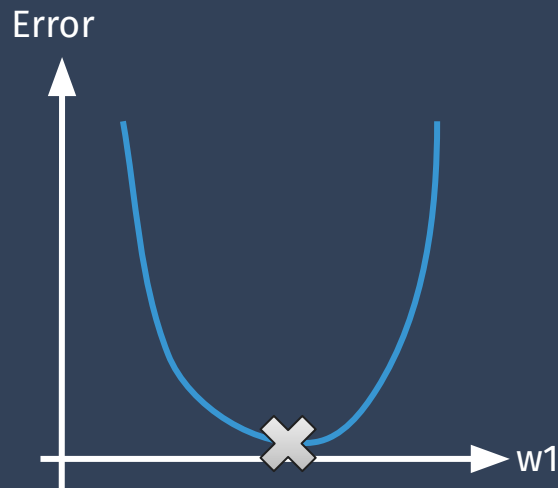
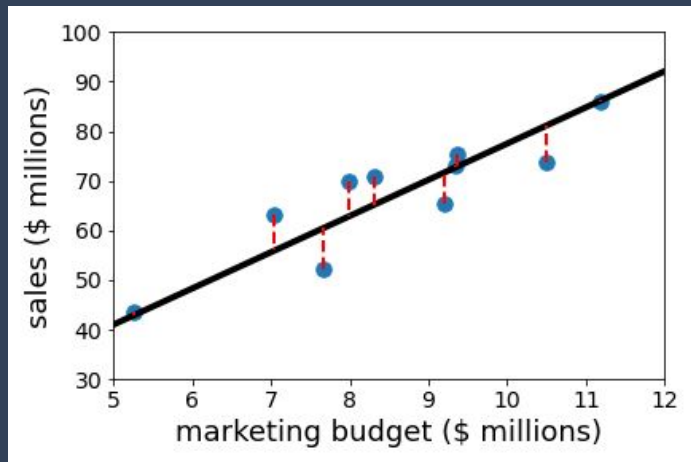
Move the line

$$Error = \frac{1}{2m} \sum_{i=1}^m (y - \hat{y})^2$$



Move the line

$$Error = \frac{1}{2m} \sum_{i=1}^m (y - \hat{y})^2$$



Analytical solution

$$\hat{y}_i = w_1 x_i + w_2$$

Analytical solution

$$\hat{y}_i = w_1 x_i + w_2$$

$$E(w_1, w_2) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2 = \frac{1}{2m} \sum_{i=1}^m (w_1 x_i + w_2 - y_i)^2$$

Analytical solution

$$\hat{y}_i = w_1 x_i + w_2$$

$$E(w_1, w_2) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2 = \frac{1}{2m} \sum_{i=1}^m (w_1 x_i + w_2 - y_i)^2$$

$$0 = \frac{\sum x_i^2}{m} w_1 + \frac{\sum x_i}{m} w_2 - \frac{\sum x_i y_i}{m}$$

$$0 = \frac{\sum x_i}{m} w_1 + w_2 - \frac{\sum y_i}{m}$$

Analytical solution

$$w_1 = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sum_{i=1}^n (X_i - \bar{X})^2}$$

$$w_2 = \bar{Y} - w_1 * \bar{X}$$

Linear Regression in sklearn

```
from sklearn.linear_model import LinearRegression
lr = LinearRegression()
lr.fit(x, y)
print(lr.coef_, lr.intercept_)
lr.predict(np.array([[6]]))
```

```
[6.29324544] 13.231274632338128
array([50.99074725])
```

Linear Regression in sklearn

```
from sklearn.linear_model import LinearRegression
lr = LinearRegression()
lr.fit(x, y)
print(lr.coef_, lr.intercept_)
lr.predict(np.array([[6]]))
```

```
[6.29324544] 13.231274632338128
array([50.99074725])
```

$\text{sales} \approx 6.29 \times \text{marketing} + 13.23$

Linear Regression in sklearn

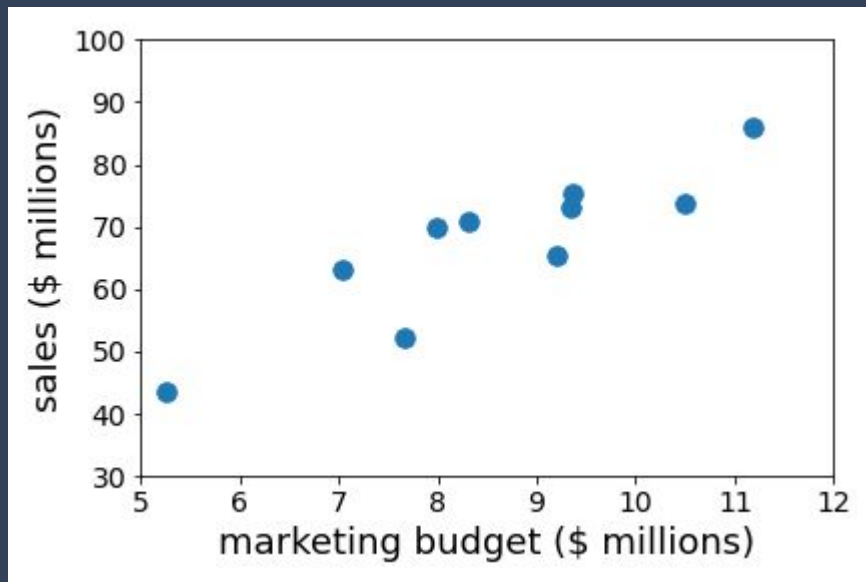
```
from sklearn.linear_model import LinearRegression
lr = LinearRegression()
lr.fit(x, y)
print(lr.coef_, lr.intercept_)
lr.predict(np.array([[6]]))

[6.29324544] 13.231274632338128
array([50.99074725])
```

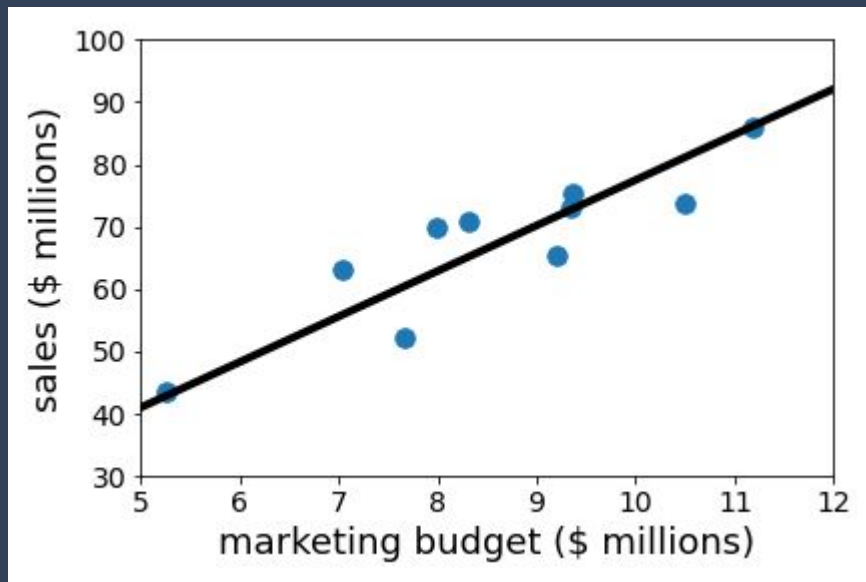
$$\text{sales} \approx 6.29 \times \text{marketing} + 13.23$$

This can be interpreted as, if we spend \$0 in marketing, we can have \$13.23 total sales. After that, every \$1 million spent on marketing should translate to a sale of \$6.29 million.

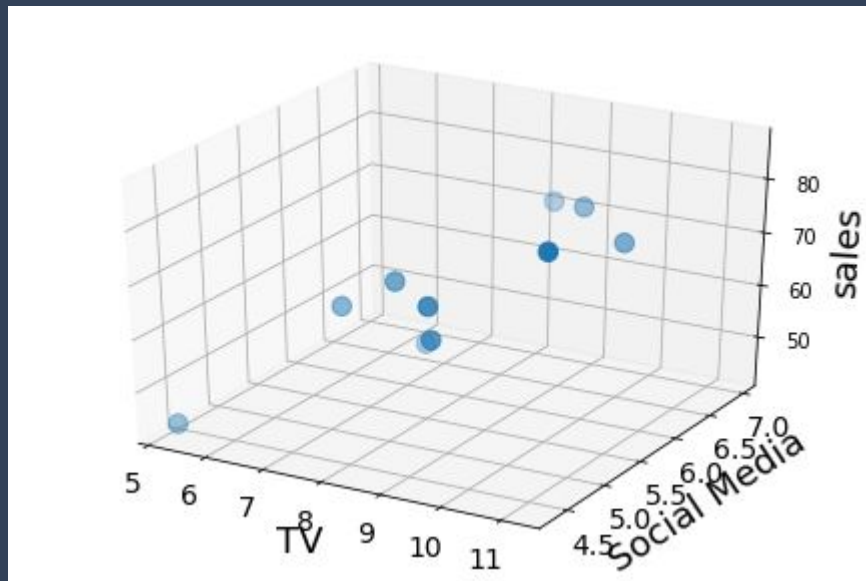
Moving to higher dimensions



Moving to higher dimensions

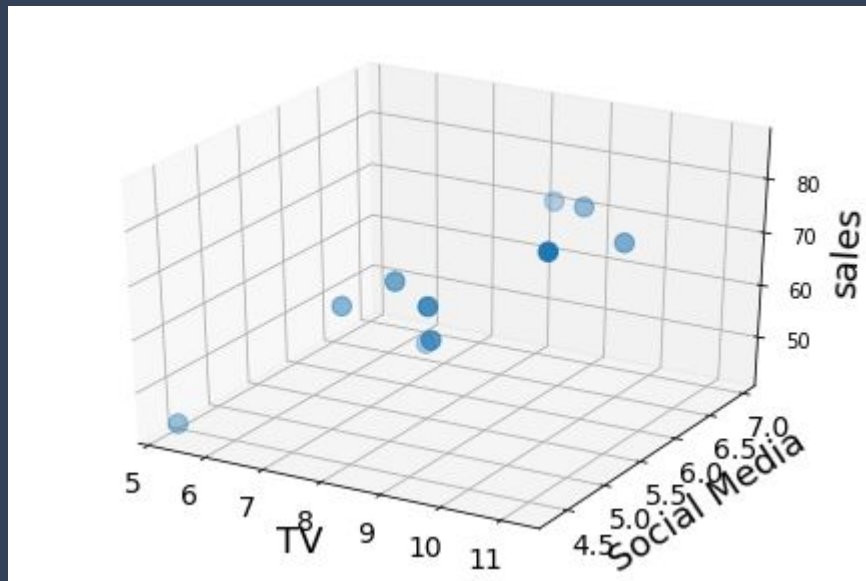


Moving to higher dimensions



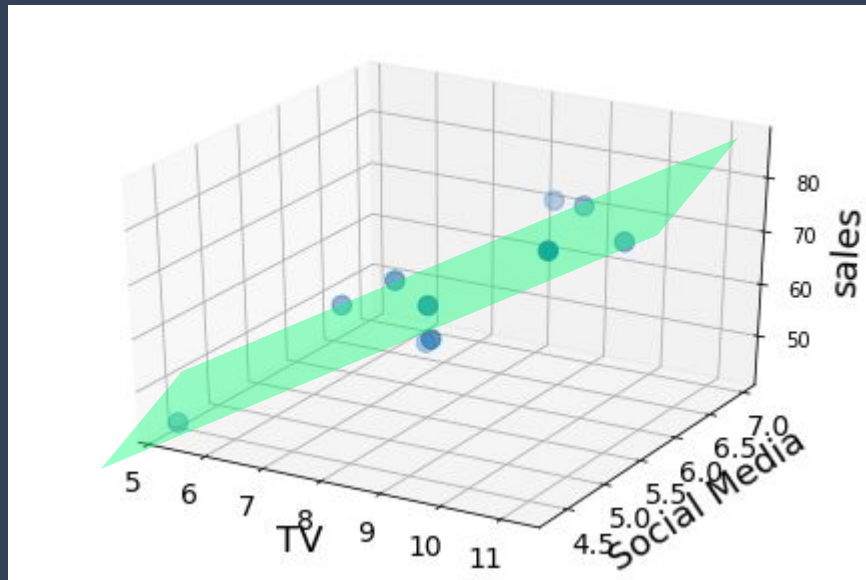
Moving to higher dimensions

$$\text{sales} \approx w_1 * M_{\text{TV}} + w_2 * M_{\text{SM}} + w_0$$



Moving to higher dimensions

$$\text{sales} \approx w_1 * M_{\text{TV}} + w_2 * M_{\text{SM}} + w_0$$



Closed form (Matrix representation)

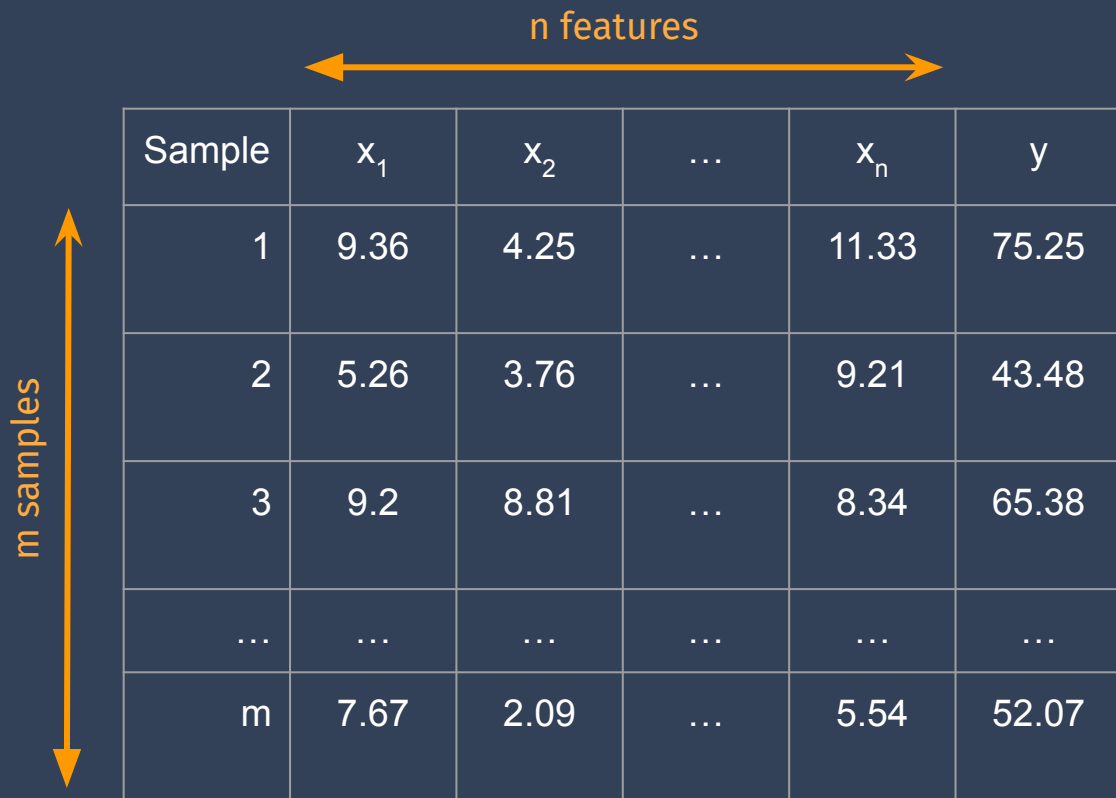
Sample	x_1	x_2	...	x_n	y
1	9.36	4.25	...	11.33	75.25
2	5.26	3.76	...	9.21	43.48
3	9.2	8.81	...	8.34	65.38
...	...	...	...	...	...
m	7.67	2.09	...	5.54	52.07

Closed form (Matrix representation)

m samples

Sample	x_1	x_2	...	x_n	y
1	9.36	4.25	...	11.33	75.25
2	5.26	3.76	...	9.21	43.48
3	9.2	8.81	...	8.34	65.38
...	...	...	...	...	...
m	7.67	2.09	...	5.54	52.07

Closed form (Matrix representation)



Sample	x_1	x_2	...	x_n	y
1	9.36	4.25	...	11.33	75.25
2	5.26	3.76	...	9.21	43.48
3	9.2	8.81	...	8.34	65.38
...	...	...	...	...	...
m	7.67	2.09	...	5.54	52.07

Matrix representation

$$X = \begin{pmatrix} x_0^{(0)} & x_1^{(0)} & \dots & x_n^{(0)} \\ x_0^{(1)} & x_1^{(1)} & \dots & x_n^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ x_0^{(m)} & x_1^{(m)} & \dots & x_n^{(m)} \end{pmatrix}$$

Matrix representation

$$X = \begin{pmatrix} x_0^{(0)} & x_1^{(0)} & \dots & x_n^{(0)} \\ x_0^{(1)} & x_1^{(1)} & \dots & x_n^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ x_0^{(m)} & x_1^{(m)} & \dots & x_n^{(m)} \end{pmatrix}$$

$$W = \begin{pmatrix} W_1 \\ W_2 \\ \vdots \\ W_n \end{pmatrix}$$

Matrix representation

$$X = \begin{pmatrix} x_0^{(0)} & x_1^{(0)} & \dots & x_n^{(0)} \\ x_0^{(1)} & x_1^{(1)} & \dots & x_n^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ x_0^{(m)} & x_1^{(m)} & \dots & x_n^{(m)} \end{pmatrix}$$

$$W = \begin{pmatrix} W_1 \\ W_2 \\ \vdots \\ W_n \end{pmatrix}$$

$$y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix}$$

Matrix representation

$$X = \begin{pmatrix} x_0^{(0)} & x_1^{(0)} & \dots & x_n^{(0)} \\ x_0^{(1)} & x_1^{(1)} & \dots & x_n^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ x_0^{(m)} & x_1^{(m)} & \dots & x_n^{(m)} \end{pmatrix}$$

$$W = \begin{pmatrix} W_1 \\ W_2 \\ \vdots \\ W_n \end{pmatrix}$$

$$y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix}$$

$$E(W) = \frac{1}{m}((XW)^T - y^T)(XW - y)$$

Matrix representation

$$X = \begin{pmatrix} x_0^{(0)} & x_1^{(0)} & \dots & x_n^{(0)} \\ x_0^{(1)} & x_1^{(1)} & \dots & x_n^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ x_0^{(m)} & x_1^{(m)} & \dots & x_n^{(m)} \end{pmatrix}$$

$$W = \begin{pmatrix} W_1 \\ W_2 \\ \vdots \\ W_n \end{pmatrix}$$

$$y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix}$$

$$E(W) = \frac{1}{m}((XW)^T - y^T)(XW - y)$$

$$\frac{\partial E}{\partial W} = 2X^T XW - 2X^T y$$

Matrix representation

$$X = \begin{pmatrix} x_0^{(0)} & x_1^{(0)} & \dots & x_n^{(0)} \\ x_0^{(1)} & x_1^{(1)} & \dots & x_n^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ x_0^{(m)} & x_1^{(m)} & \dots & x_n^{(m)} \end{pmatrix}$$

$$W = \begin{pmatrix} W_1 \\ W_2 \\ \vdots \\ W_n \end{pmatrix}$$

$$y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix}$$

$$E(W) = \frac{1}{m}((XW)^T - y^T)(XW - y)$$

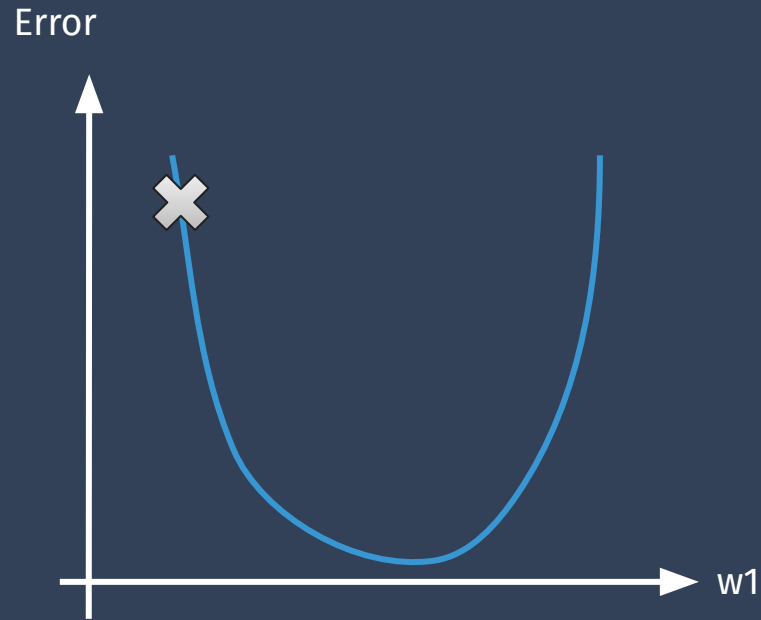
$$\frac{\partial E}{\partial W} = 2X^T XW - 2X^T y$$

$$W = (X^T X)^{-1} X^T y$$

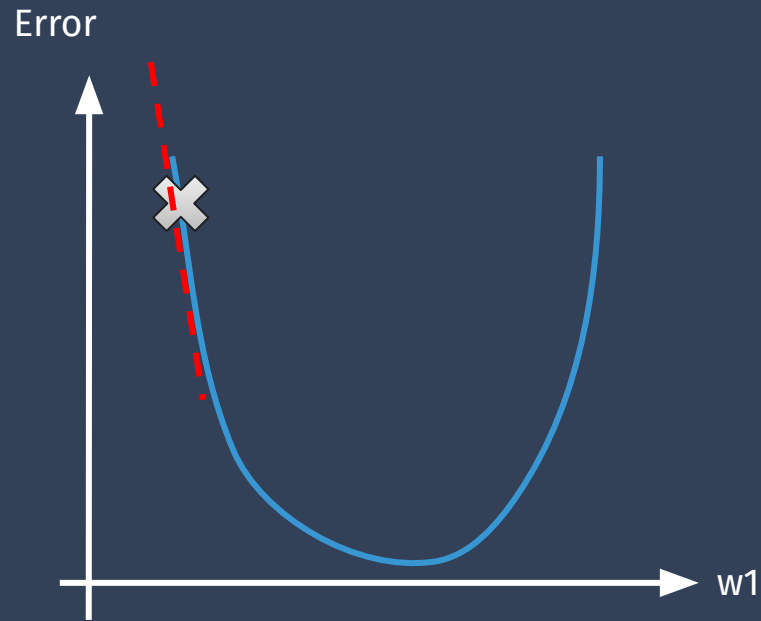
Problem with the Closed Form

- Matrix multiplication is fast (with software/hardware optimizations), but requires lots of memory for large datasets

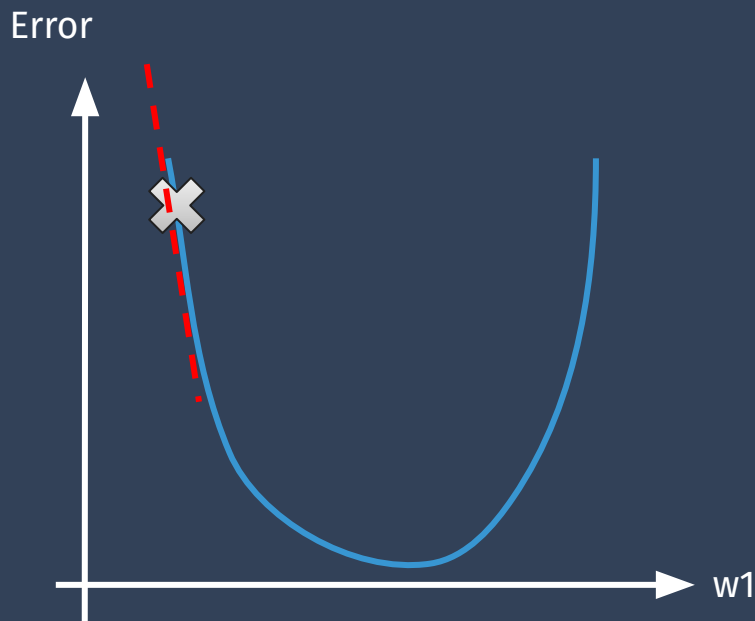
Gradient Descent



Gradient Descent

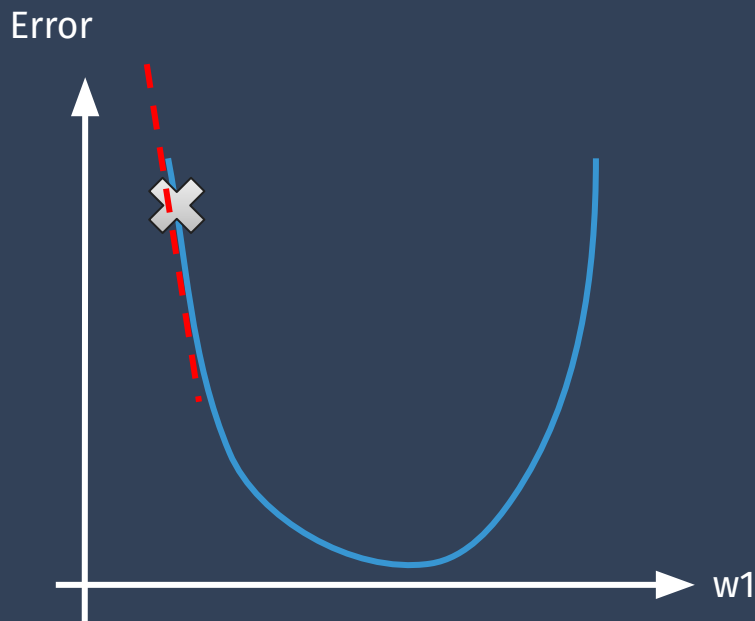


Gradient Descent



$$w_i \rightarrow w_i - \alpha \frac{\partial}{\partial w_i} \text{Error}$$

Gradient Descent



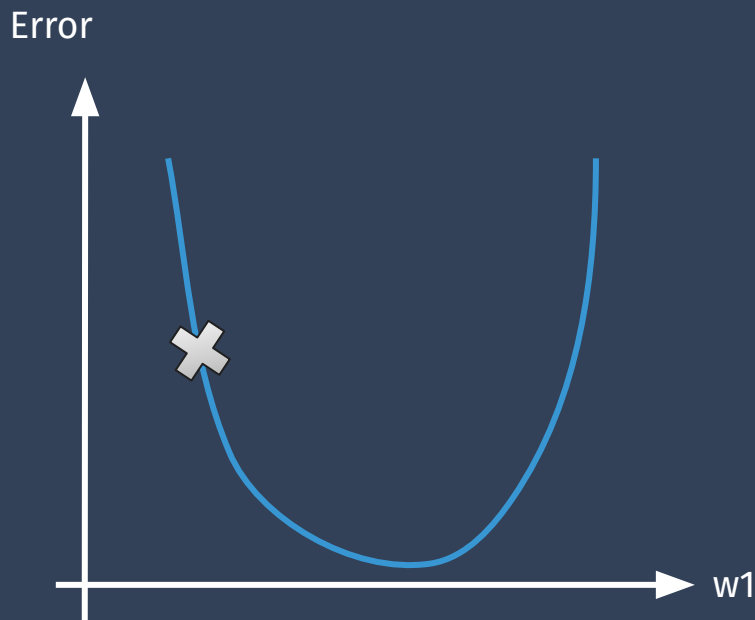
$$w_i \rightarrow w_i - \alpha \frac{\partial}{\partial w_i} \text{Error}$$

$$\text{Error} = \frac{1}{2}(y - \hat{y})^2$$

$$\frac{\partial}{\partial w_1} \text{Error} = -(y - \hat{y}) x$$

$$\frac{\partial}{\partial w_2} \text{Error} = -(y - \hat{y})$$

Gradient Descent



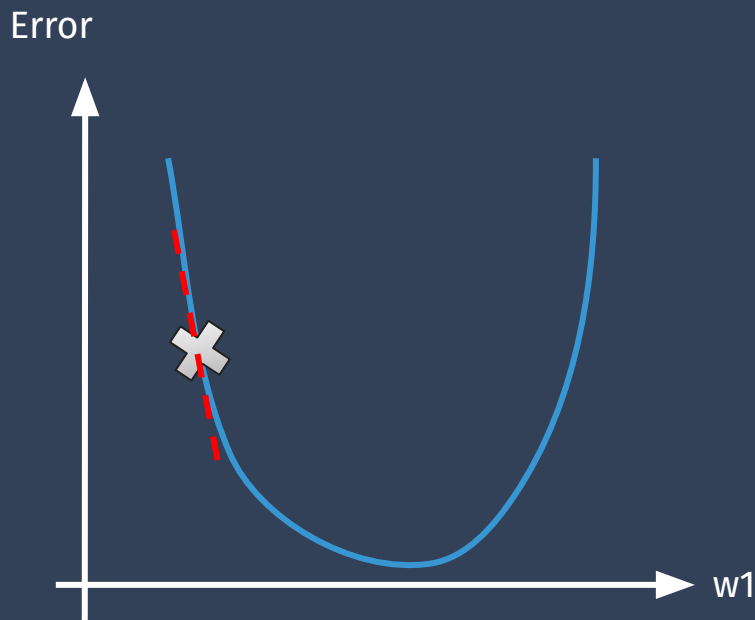
$$w_i \rightarrow w_i - \alpha \frac{\partial}{\partial w_i} \text{Error}$$

$$\text{Error} = \frac{1}{2}(y - \hat{y})^2$$

$$\frac{\partial}{\partial w_1} \text{Error} = -(y - \hat{y}) x$$

$$\frac{\partial}{\partial w_2} \text{Error} = -(y - \hat{y})$$

Gradient Descent



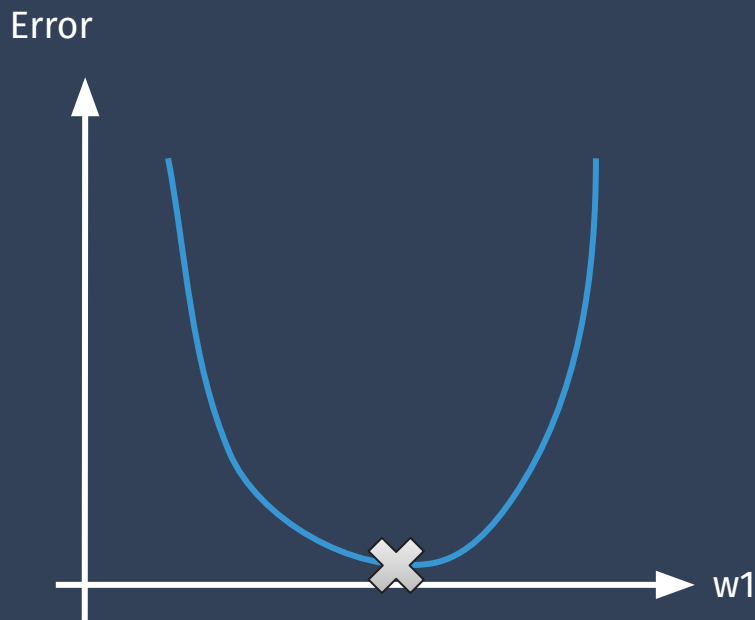
$$w_i \rightarrow w_i - \alpha \frac{\partial}{\partial w_i} \text{Error}$$

$$\text{Error} = \frac{1}{2}(y - \hat{y})^2$$

$$\frac{\partial}{\partial w_1} \text{Error} = -(y - \hat{y}) x$$

$$\frac{\partial}{\partial w_2} \text{Error} = -(y - \hat{y})$$

Gradient Descent



$$w_i \rightarrow w_i - \alpha \frac{\partial}{\partial w_i} \text{Error}$$

$$\text{Error} = \frac{1}{2}(y - \hat{y})^2$$

$$\frac{\partial}{\partial w_1} \text{Error} = -(y - \hat{y}) x$$

$$\frac{\partial}{\partial w_2} \text{Error} = -(y - \hat{y})$$

Closed form vs GD

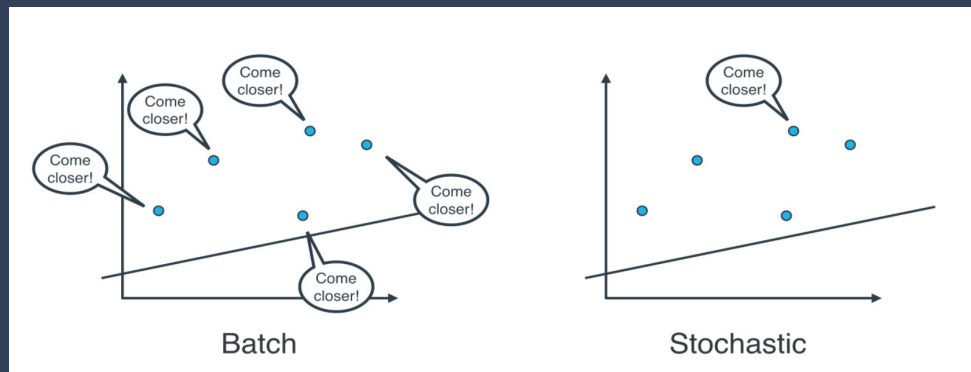
- Matrix multiplication is fast (with software/hardware optimizations), but requires lots of memory for large datasets
- GD allows us to repeatedly use a subset of the training data (as small as just 1 sample).

Closed form vs GD

- Matrix multiplication is fast (with software/hardware optimizations), but requires lots of memory for large datasets
- GD allows us to repeatedly use a subset of the training data (as small as just 1 sample).
 - Batch GD
 - Stochastic GD (SGD)
 - Mini-batch GD

Closed form vs GD

- Matrix multiplication is fast (with software/hardware optimizations), but requires lots of memory for large datasets
- GD allows us to repeatedly use a subset of the training data (as small as just 1 sample).
 - Batch GD
 - Stochastic GD (SGD)
 - Mini-batch GD



Interpreting the results

Plugging in our training data, we get

$$\text{sales} \approx 1.88 * M_{\text{TV}} + 4.23 * M_{\text{SM}} + 9.97$$



INTERVIEW
KICKSTART

Interpreting the results

Plugging in our training data, we get

$$\text{sales} \approx 1.88 * M_{\text{TV}} + 4.23 * M_{\text{SM}} + 9.97$$

- How sales respond to each channel of marketing (for every dollar spent)

Interpreting the results

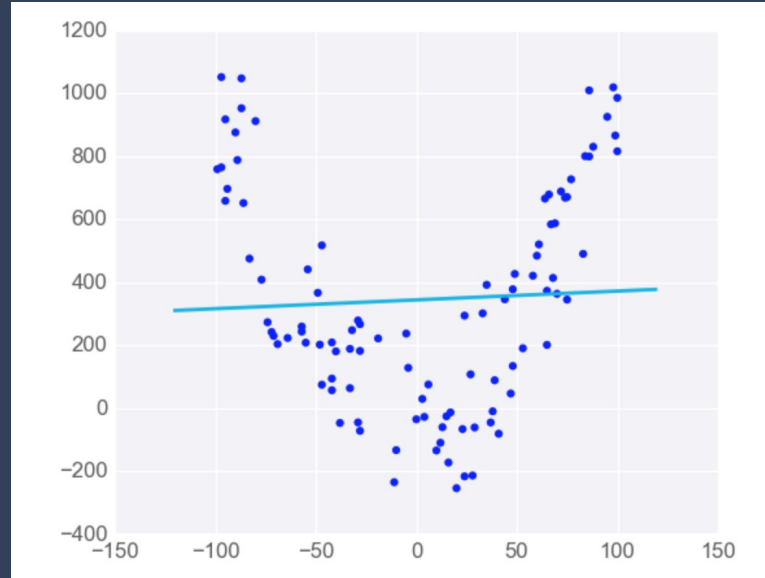
Plugging in our training data, we get

$$\text{sales} \approx 1.88 * M_{TV} + 4.23 * M_{SM} + 9.97$$

- How sales respond to each channel of marketing (for every dollar spent)
- Statistical significance of the “linear relations” through the regression - p-values
 - Null hypothesis: “the coefficient is 0 for each independent variable”
 - Each coefficient comes with a p-value. The smaller it is, the more confident we are to reject the null (rule of thumb < 0.05).

Linear Regression assumptions

- Linearity



Linear Regression assumptions

- Linearity
- Normality

Linear Regression assumptions

- Linearity
- Normality
- Non-Collinearity

Linear Regression assumptions

- Linearity
- Normality
- Non-Collinearity
 - Univariate: Pearson correlation map

Linear Regression assumptions

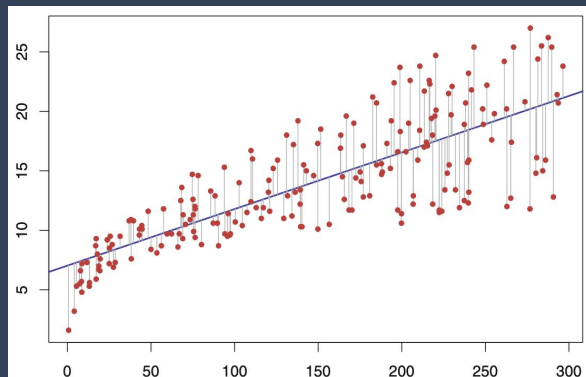
- Linearity
- Normality
- Non-Collinearity
 - Univariate: Pearson correlation map
 - Multicollinearity: Variance Inflation Factor

Linear Regression assumptions

- Linearity
- Normality
- Non-Collinearity
 - Univariate: Pearson correlation map
 - Multicollinearity: Variance Inflation Factor
- Homoscedasticity

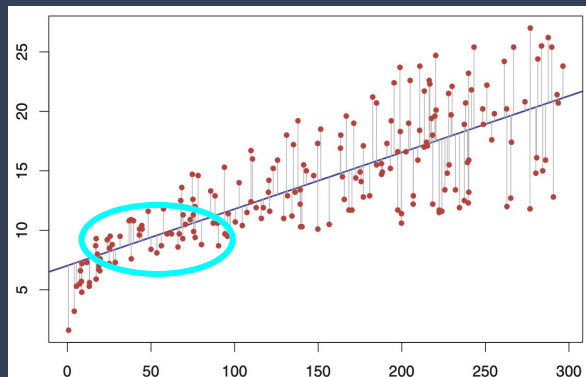
Linear Regression assumptions

- Linearity
- Normality
- Non-Collinearity
 - Univariate: Pearson correlation map
 - Multicollinearity: Variance Inflation Factor
- Homoscedasticity



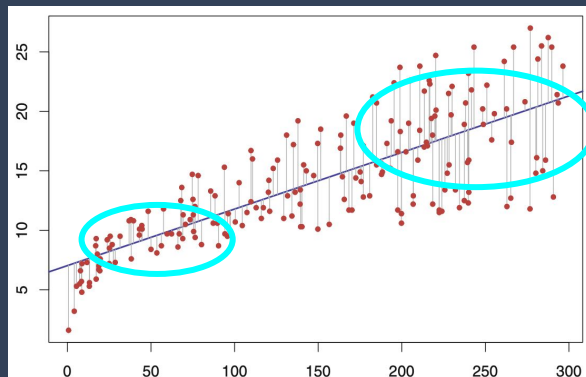
Linear Regression assumptions

- Linearity
- Normality
- Non-Collinearity
 - Univariate: Pearson correlation map
 - Multicollinearity: Variance Inflation Factor
- Homoscedasticity

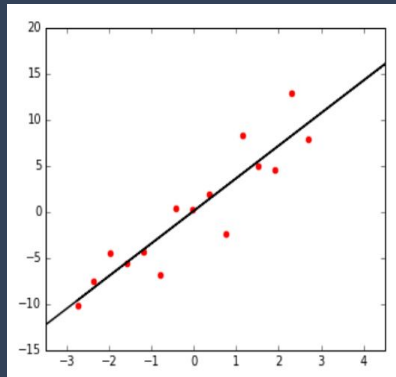


Linear Regression assumptions

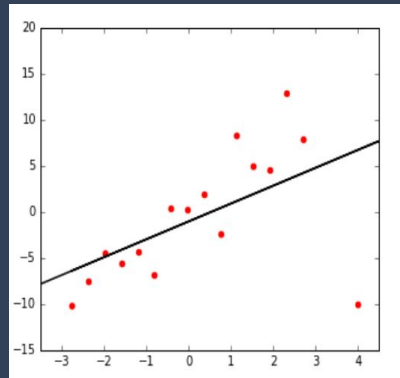
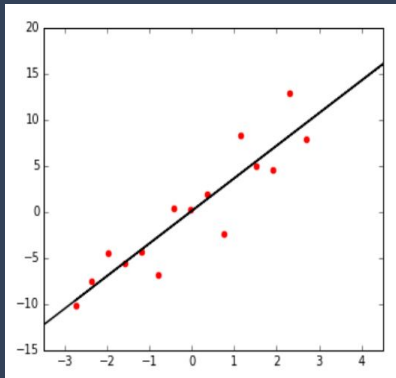
- Linearity
- Normality
- Non-Collinearity
 - Univariate: Pearson correlation map
 - Multicollinearity: Variance Inflation Factor
- Homoscedasticity



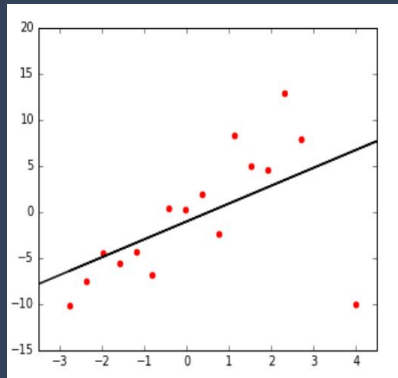
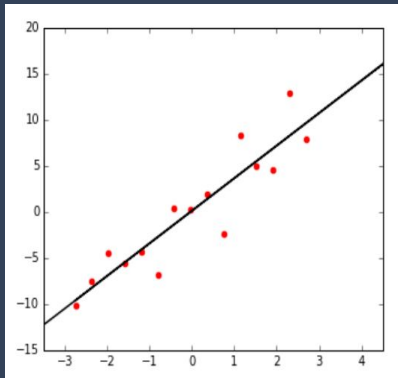
Outliers



Outliers

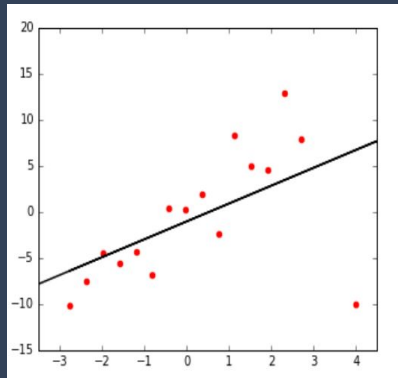
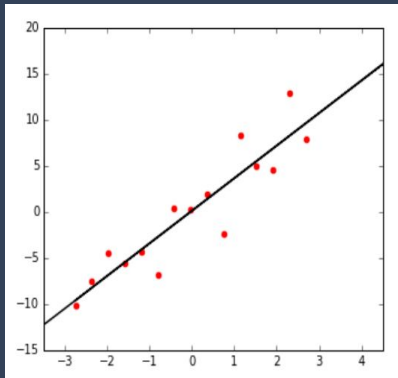


Outliers



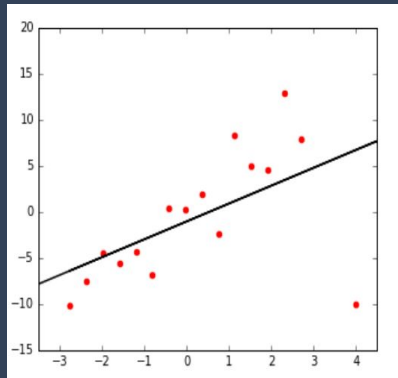
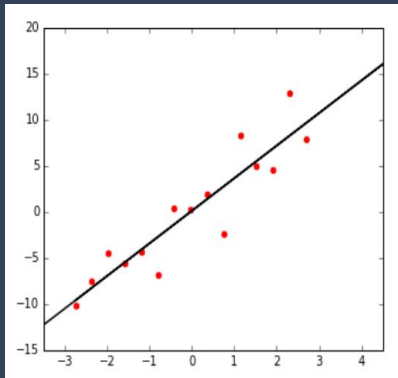
- Do they matter?

Outliers



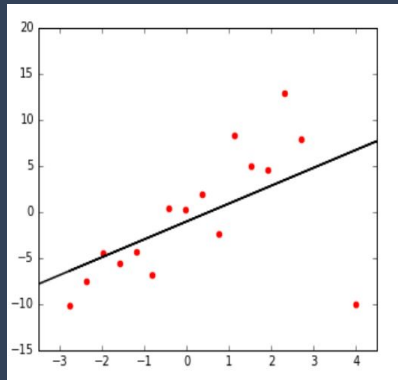
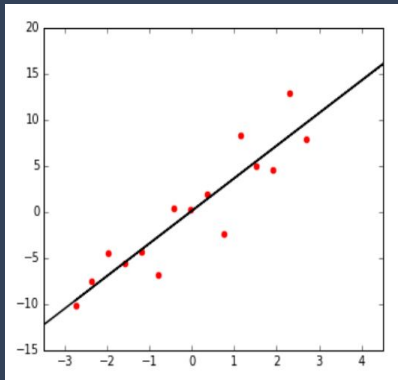
- Do they matter?
Yes! Outliers respond to the loss function therefore the regression results significantly.

Outliers



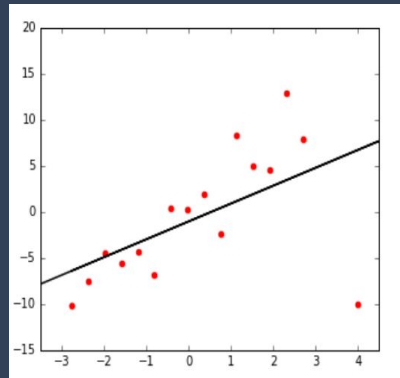
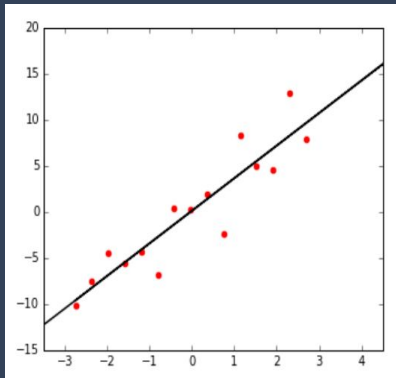
- Do they matter?
Yes! Outliers respond to the loss function therefore the regression results significantly.
- How do detect them?

Outliers



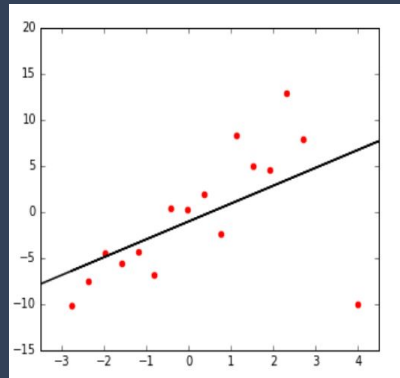
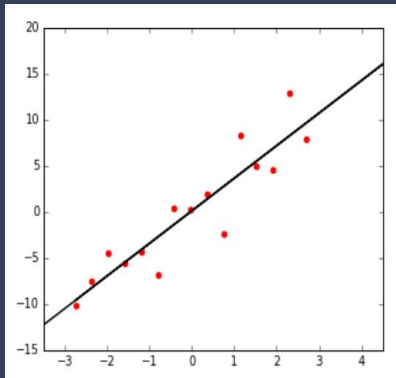
- Do they matter?
Yes! Outliers respond to the loss function therefore the regression results significantly.
- How do detect them?
z-score, DBscan, isolated trees...

Outliers



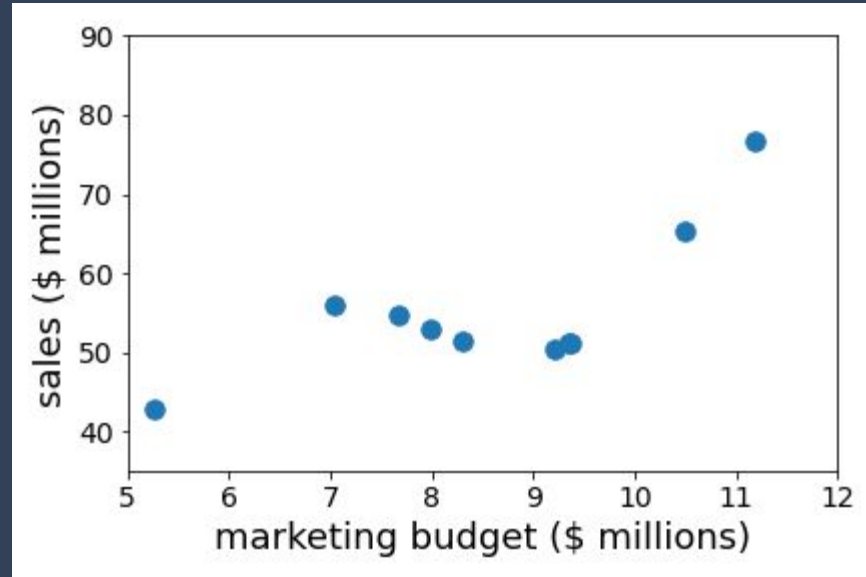
- Do they matter?
Yes! Outliers respond to the loss function therefore the regression results significantly.
- How do detect them?
z-score, DBscan, isolated trees...
- Do we always find and remove the outliers?

Outliers



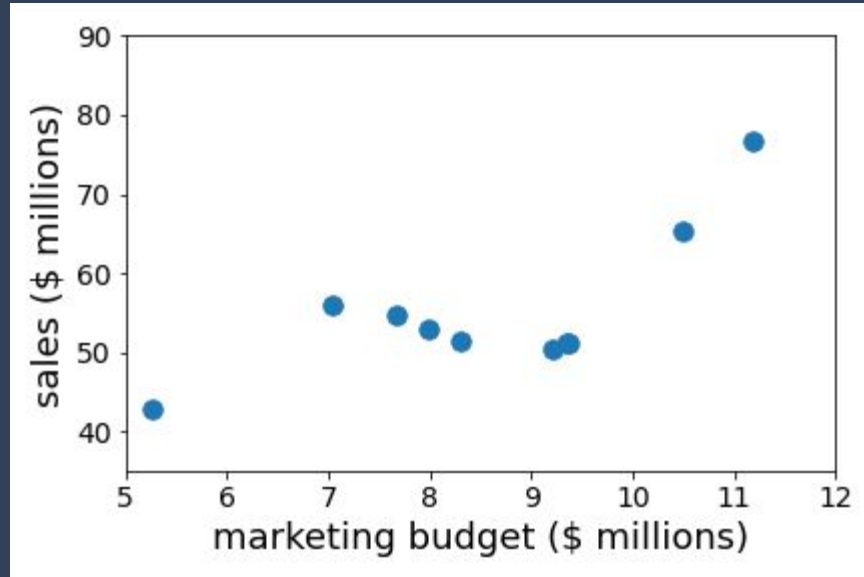
- Do they matter?
Yes! Outliers respond to the loss function therefore the regression results significantly.
- How do detect them?
z-score, DBscan, isolated trees...
- Do we always find and remove the outliers?
Depends on the system

Polynomial Regression



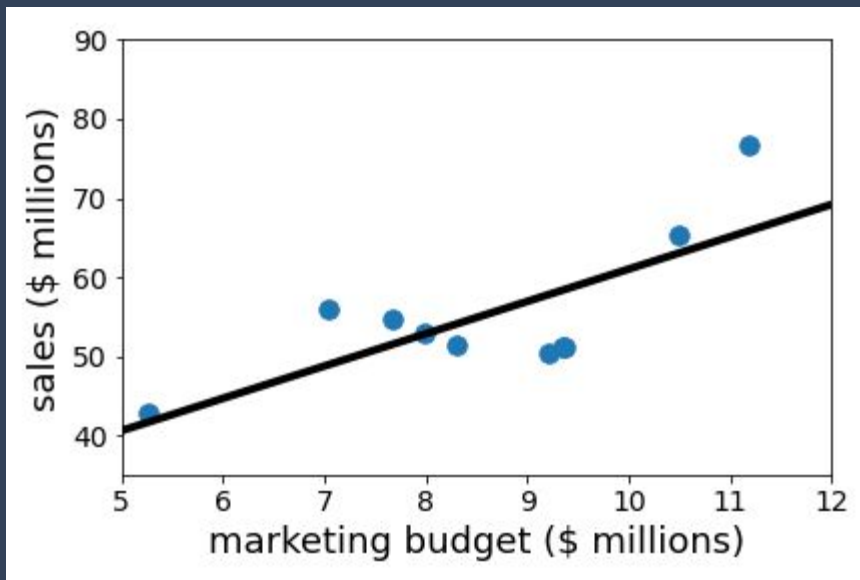
Polynomial Regression

$$y = w_1 * x + w_0$$



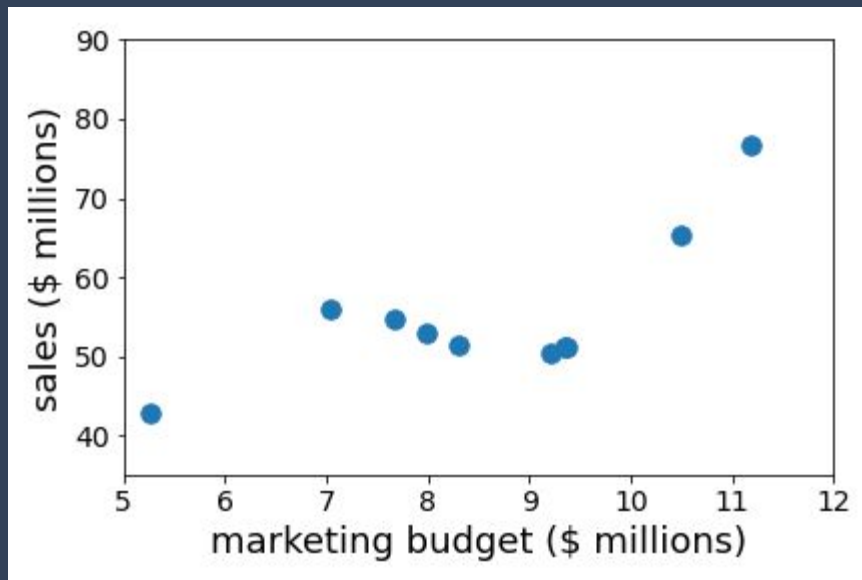
Polynomial Regression

$$y = w_1 * x + w_0$$



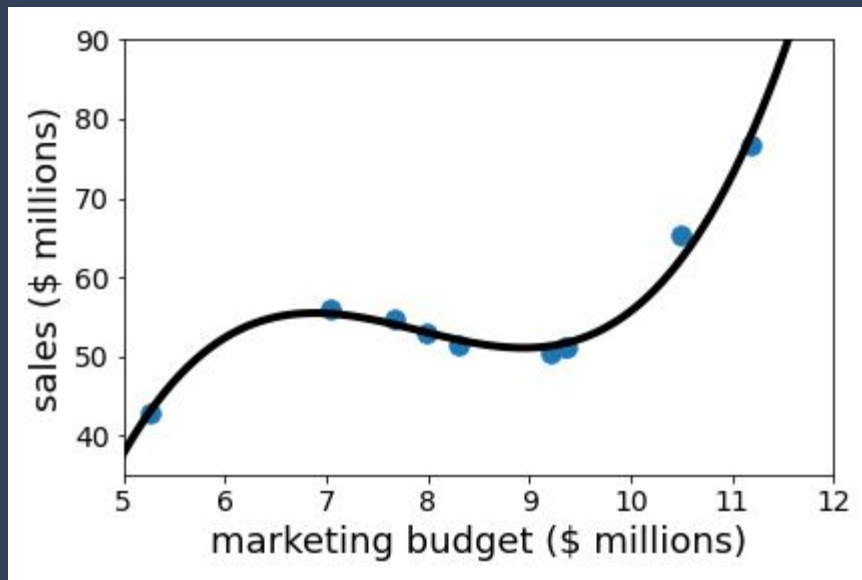
Polynomial Regression

$$y = w_3 * x^3 + w_2 * x^2 + w_1 * x + w_0$$

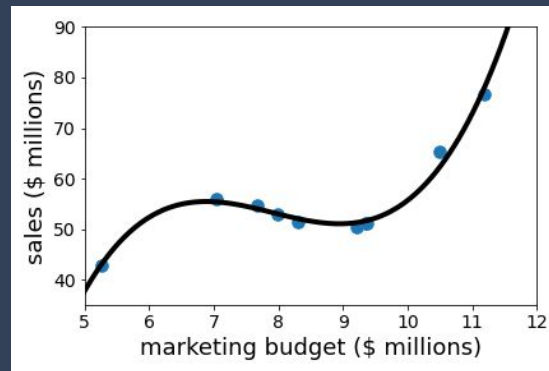
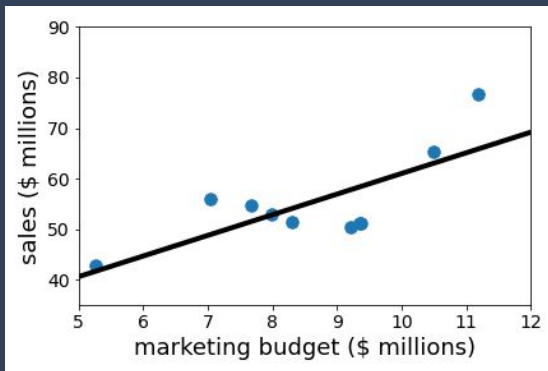


Polynomial Regression

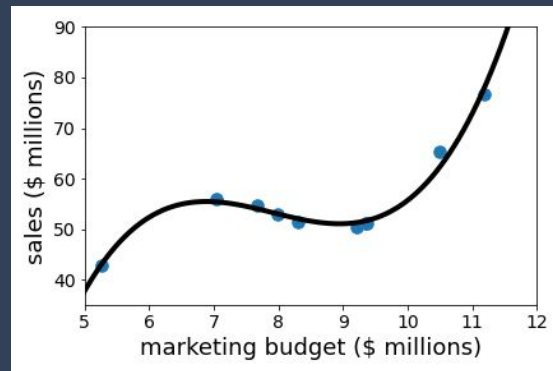
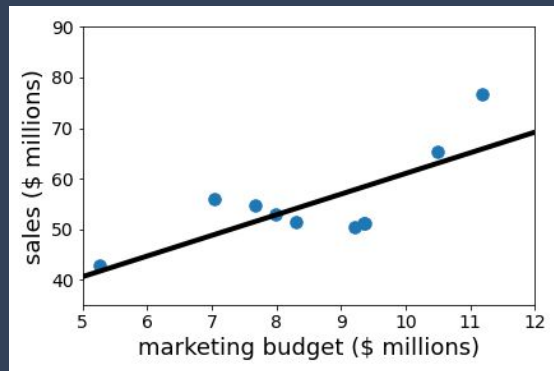
$$y = w_3 * x^3 + w_2 * x^2 + w_1 * x + w_0$$



Bias Variance Tradeoff

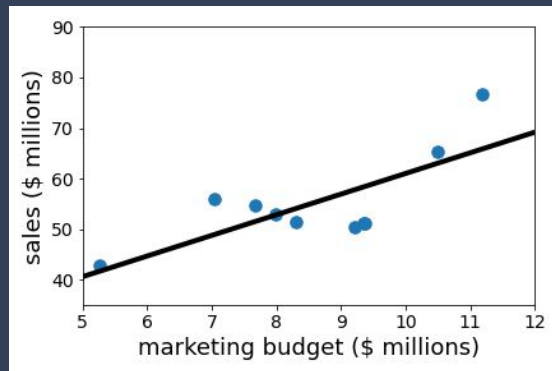


Bias Variance Tradeoff

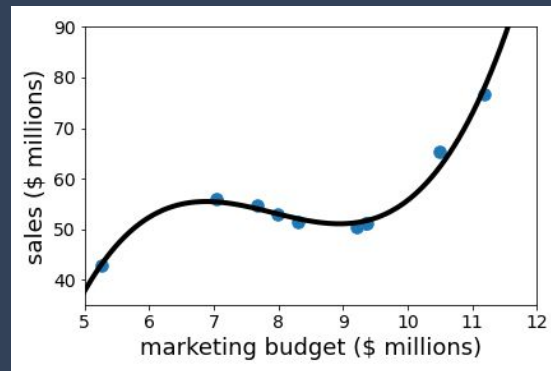


Complexity

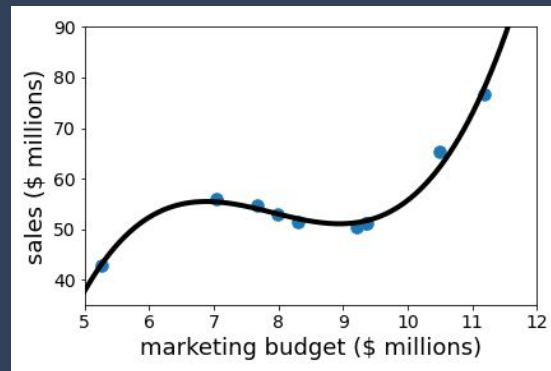
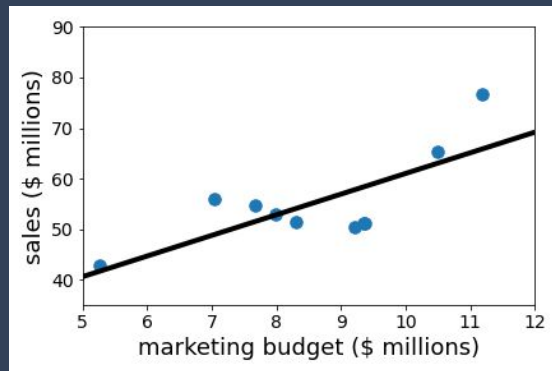
Bias Variance Tradeoff



Complexity



Bias Variance Tradeoff

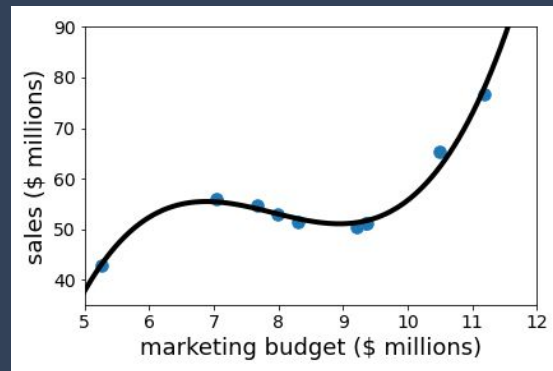
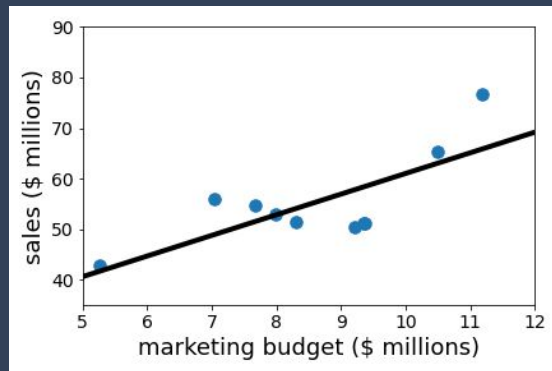


Complexity



Bias

Bias Variance Tradeoff



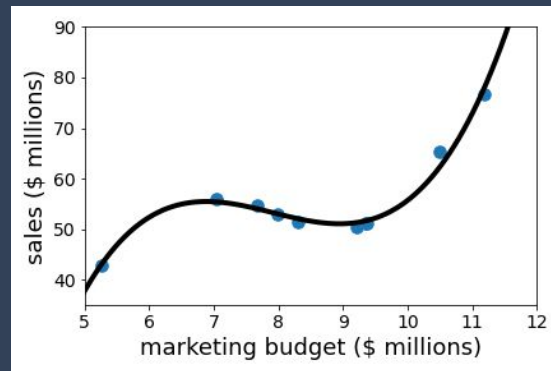
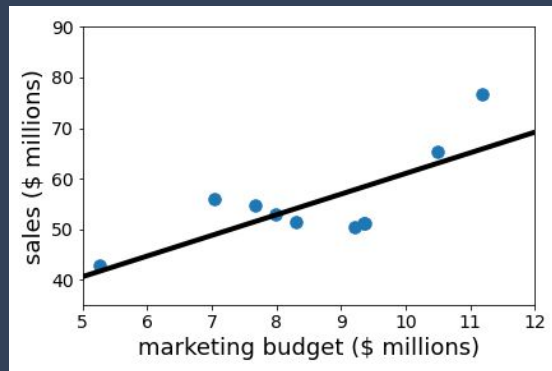
Complexity



Bias



Bias Variance Tradeoff



Complexity

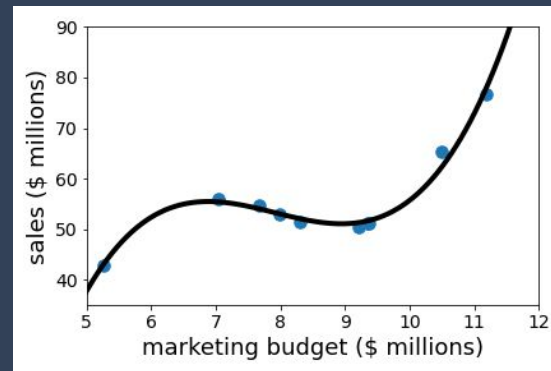
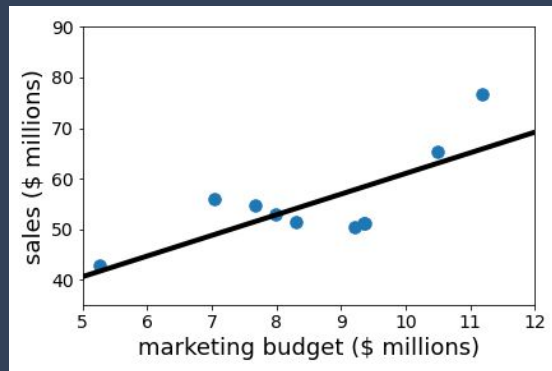


Bias



Variance

Bias Variance Tradeoff



Complexity



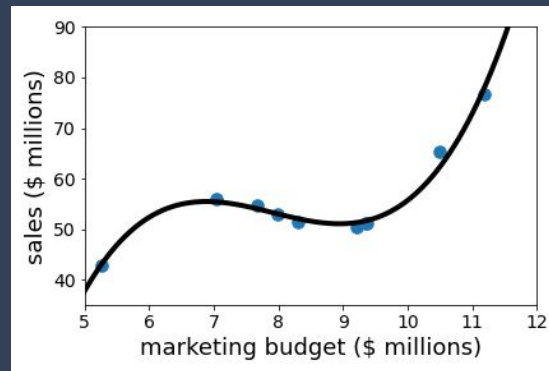
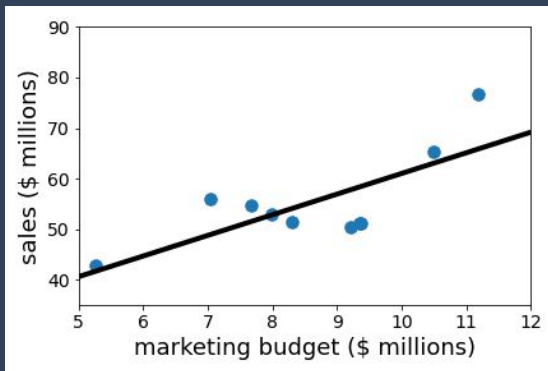
Bias



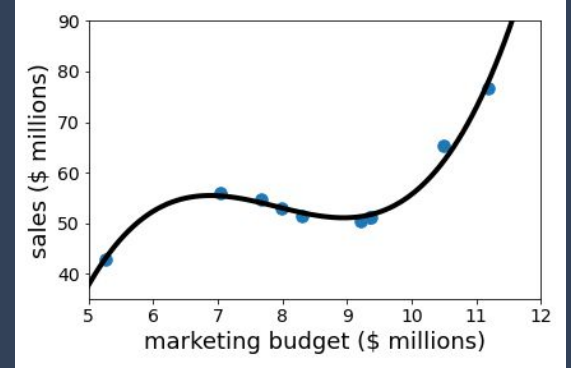
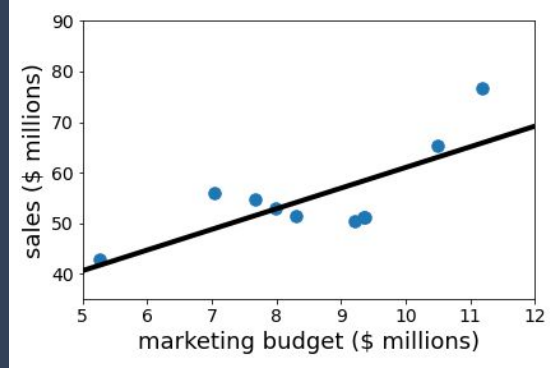
Variance



Regularization



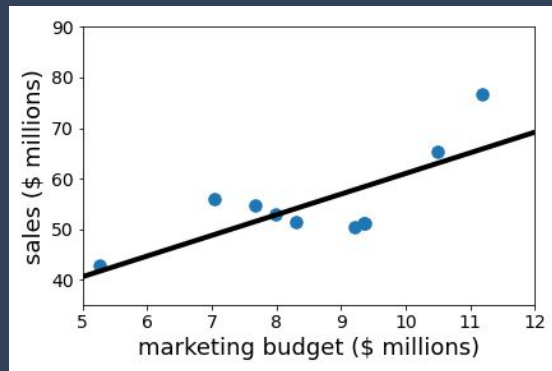
Regularization



Error

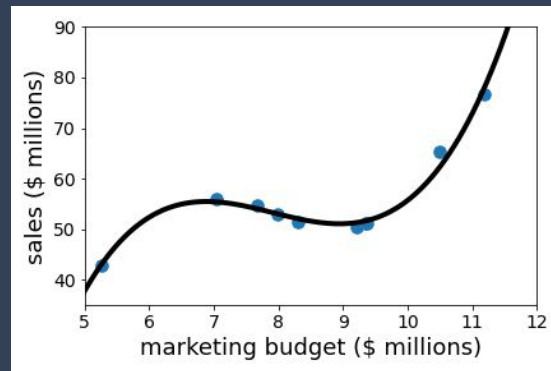
$$y = 4.1 * x + 22.1$$

Regularization



Error

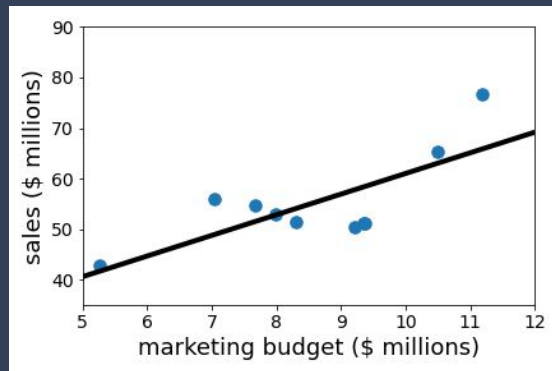
$$y = 4.1 * x + 22.1$$



Error

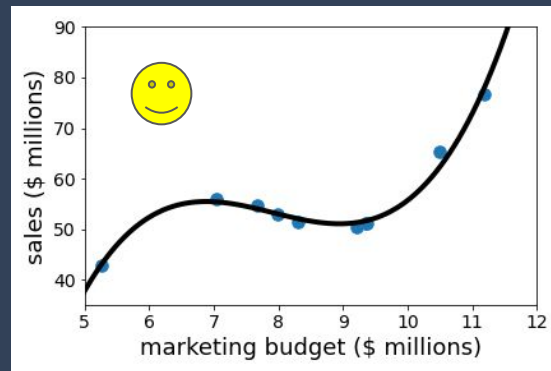
$$y = 1.1 * x^3 - 23.8 * x^2 + 185 * x - 419$$

Regularization



Error

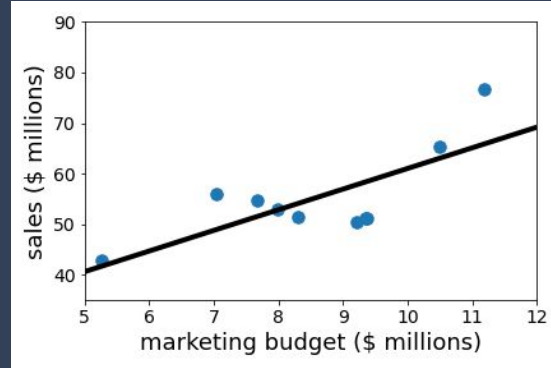
$$y = 4.1 * x + 22.1$$



Error

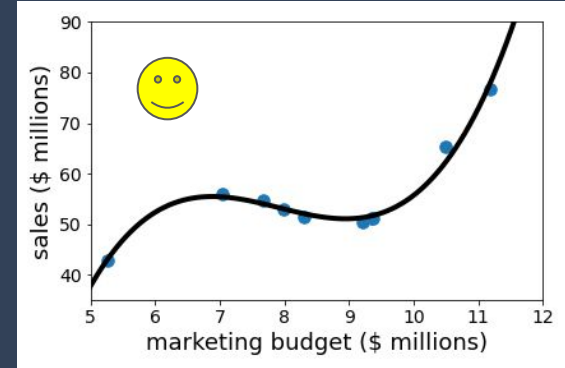
$$y = 1.1 * x^3 - 23.8 * x^2 + 185 * x - 419$$

Regularization



Error

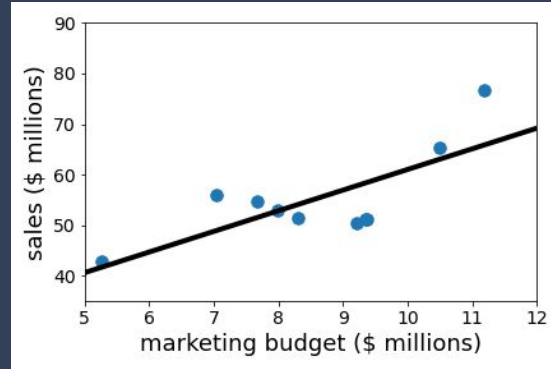
$$y = 4.1x + 22.1$$



Error

$$y = 1.1x^3 - 23.8x^2 + 185x - 419$$

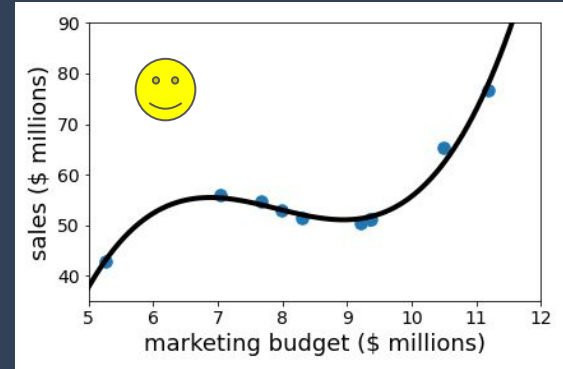
Regularization



Error



$$y = 4.1x + 22.1$$

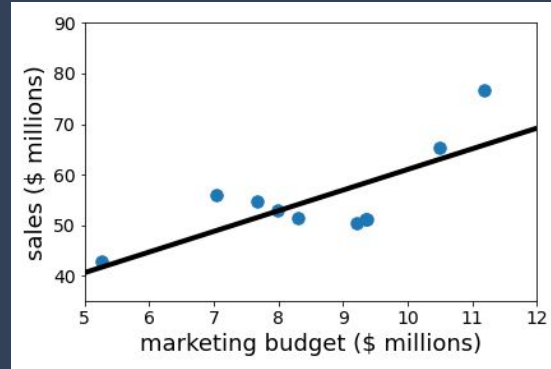


Error



$$y = 1.1x^3 - 23.8x^2 + 185x - 419$$

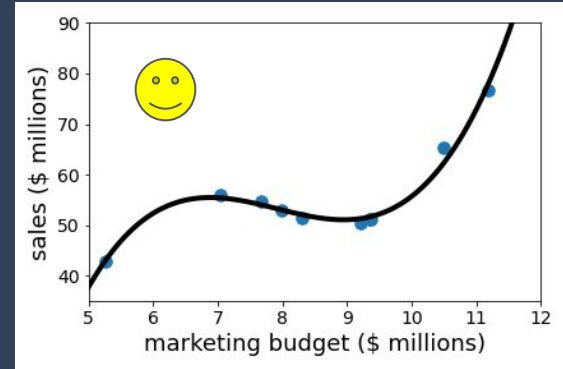
Regularization



Error



$$y = 4.1x + 22.1$$

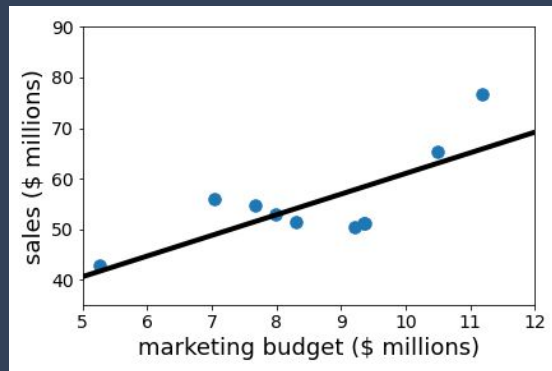


Error



$$y = 1.1x^3 - 23.8x^2 + 185x - 419$$

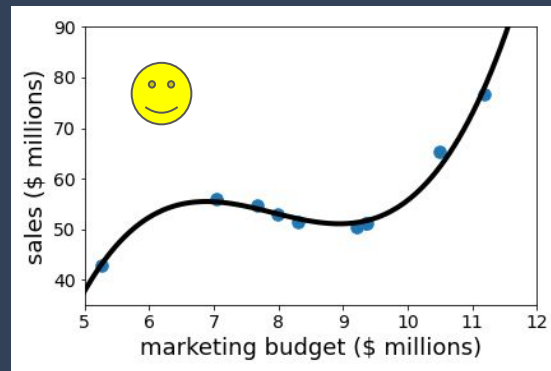
Regularization



Error



$$y = 4.1x + 22.1$$

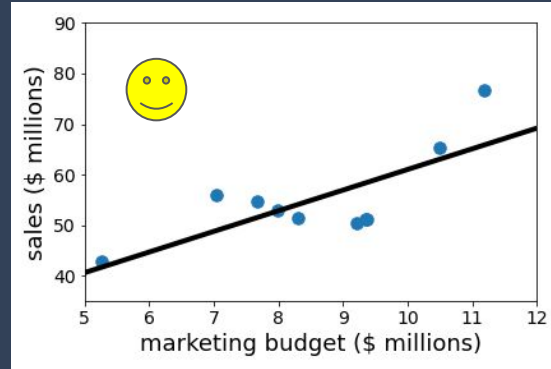


Error



$$y = 1.1x^3 - 23.8x^2 + 185x - 419$$

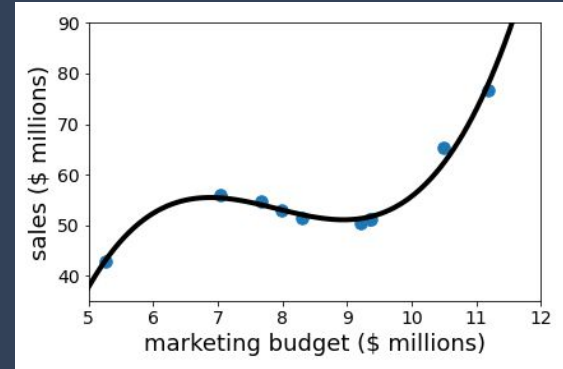
Regularization



Error



$$y = 4.1x + 22.1$$

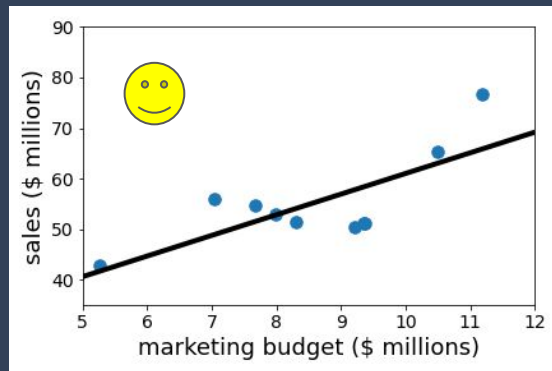


Error



$$y = 1.1x^3 - 23.8x^2 + 185x - 419$$

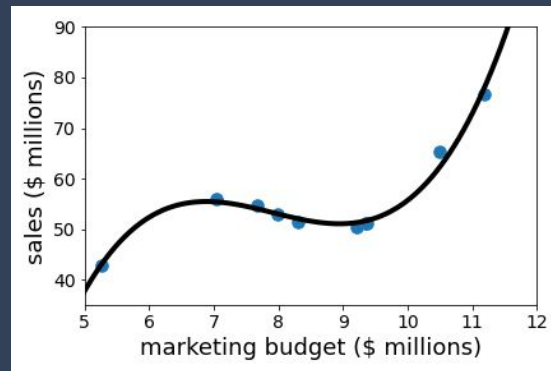
Regularization



Error



$$y = 4.1x + 22.1$$



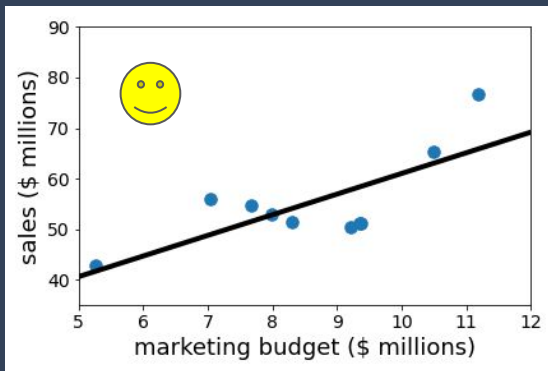
Error



$$y = 1.1x^3 - 23.8x^2 + 185x - 419$$

$$\text{Ridge: Loss}(w) = \text{MSE} + \lambda^* ||w||^2$$

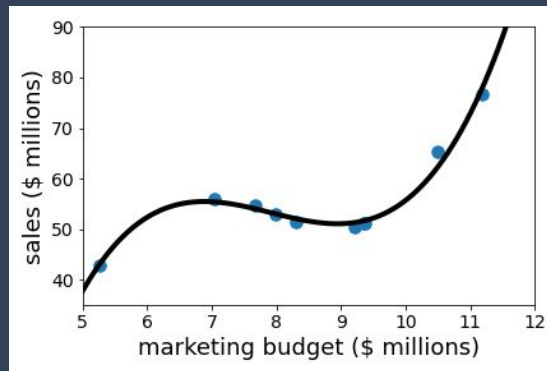
Regularization



Error



$$y = 4.1x + 22.1$$



Error



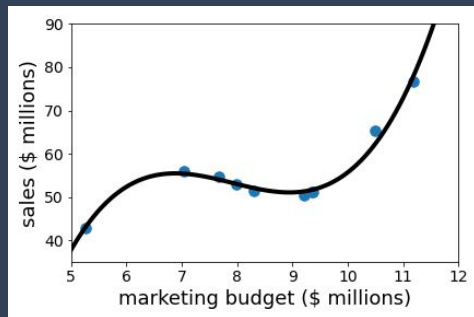
$$y = 1.1x^3 - 23.8x^2 + 185x - 419$$

$$\text{Ridge: Loss}(w) = \text{MSE} + \lambda^* ||w||^2$$

$$\text{Lasso: Loss}(w) = \text{MSE} + \lambda^* ||w||$$

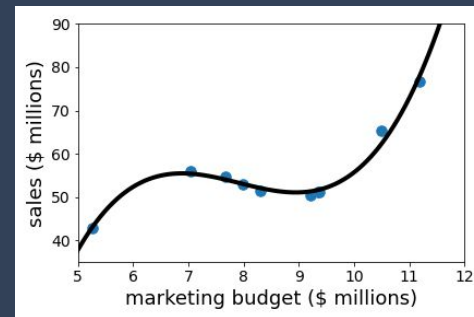
Ridge vs Lasso

Ridge Regression



$$y = 1.1x^3 - 23.8x^2 + 185x - 419$$

Lasso Regression

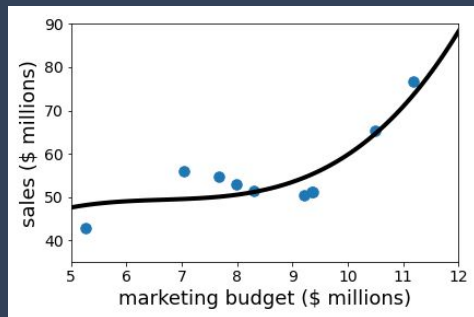


$$y = 1.1x^3 - 23.8x^2 + 185x - 419$$

$\lambda = 0$

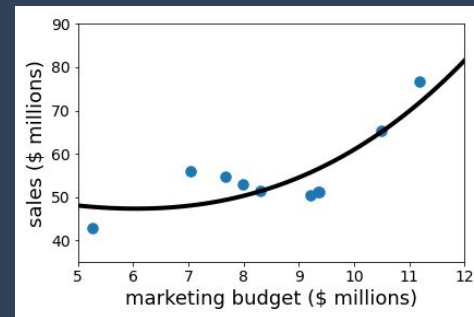
Ridge vs Lasso

Ridge Regression



$$y = 0.23x^3 - 4.8x^2 + 32.1x - 23.8$$

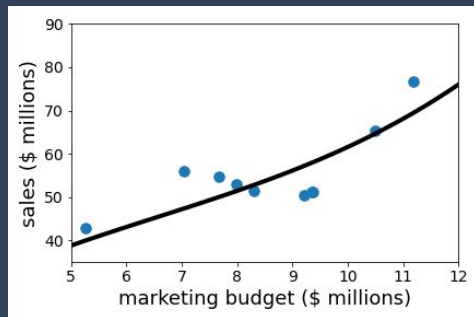
Lasso Regression



$$y = 0.04x^3 - 0.15x^2 - 3.26x + 6.24$$

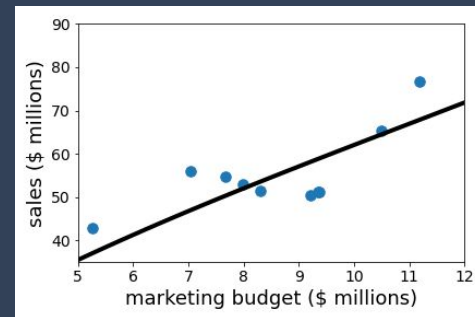
Ridge vs Lasso

Ridge Regression



$$y = 0.05x^3 - 1.1x^2 + 10.8x - 3.8$$

Lasso Regression



$$y = 0.01x^3 + 8.5x^2 - 0.33x + 0.0$$

Regularization

What about multi linear regression?

Regularization

What about multi linear regression?

$$y = w_1 * x_1 + w_2 * x_2 + w_3 * x_3 + \dots + w_n * x_n + w_0$$

Regularization

What about multi linear regression?

$$y = w_1^* x_1 + w_2^* x_2 + w_3^* x_3 + \dots + w_n^* x_n + w_0$$

Ridge: $\text{Loss}(w) = \text{MSE} + \lambda^* ||w||^2$, all w reduce in size

Regularization

What about multi linear regression?

$$y = w_1^* x_1 + w_2^* x_2 + w_3^* x_3 + \dots + w_n^* x_n + w_0$$

Ridge: $\text{Loss}(w) = \text{MSE} + \lambda^* ||w||^2$, all w reduce in size

Lasso: $\text{Loss}(w) = \text{MSE} + \lambda^* ||w||$, some w goes to zero

Regularization

- Regularization reduces the variance of the model by adding Lasso (L1) or Ridge (L2) regularization terms into the loss function.

Regularization

- Regularization reduces the variance of the model by adding Lasso (L1) or Ridge (L2) regularization terms into the loss function.
- With larger regularization strength (λ coefficient), L2 regularization tends to push the magnitude of all the coefficients smaller; while L1 regularization pushes some of the coefficients close to zero. This is why L1 regularization can be used in feature selection.

Regularization

- Regularization reduces the variance of the model by adding Lasso (L1) or Ridge (L2) regularization terms into the loss function.
- With larger regularization strength (λ coefficient), L2 regularization tends to push the magnitude of all the coefficients smaller; while L1 regularization pushes some of the coefficients close to zero. This is why L1 regularization can be used in feature selection.
- Regularization helps with reducing potential overfitting. However, if the model was originally underfit (with high bias), applying regularization would make it worse.

Regularization

- Regularization reduces the variance of the model by adding Lasso (L1) or Ridge (L2) regularization terms into the loss function.
- With larger regularization strength (λ coefficient), L2 regularization tends to push the magnitude of all the coefficients smaller; while L1 regularization pushes some of the coefficients close to zero. This is why L1 regularization can be used in feature selection.
- Regularization helps with reducing potential overfitting. However, if the model was originally underfit (with high bias), applying regularization would make it worse.
- The regularization strength coefficient (λ), can be treated as a model hyperparameter and be tuned by the validation dataset metrics.

Metrics

Metrics

- MSE/RMSE
 - sensitive to the outliers
 - large error terms

Metrics

- MSE/RMSE
 - sensitive to the outliers
 - large error terms
- MAE

Metrics

- MSE/RMSE
 - sensitive to the outliers
 - large error terms
- MAE
- MAPE

Metrics

- MSE/RMSE
 - sensitive to the outliers
 - large error terms
- MAE
- MAPE
- R^2 /adjusted R^2
 - measures correlation only

Metrics

- MSE/RMSE
 - sensitive to the outliers
 - large error terms
- MAE
- MAPE
- R^2 /adjusted R^2
 - measures correlation only
- MASE